

CSE 385: Analysis of Algorithms (Honors)

Lecture 10 (Deterministic Quicksort and its Average Case Analysis)

**Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026**

The Divide-and-Conquer Process in Merge Sort

Suppose we want to sort a typical subarray $A[p..r]$.

DIVIDE: Split $A[p..r]$ at midpoint q into two subarrays $A[p..q]$ and $A[q + 1..r]$ of equal or almost equal length.

CONQUER: Recursively sort $A[p..q]$ and $A[q + 1..r]$.

COMBINE: Merge the two sorted subarrays $A[p..q]$ and $A[q + 1..r]$ to obtain a longer sorted subarray $A[p..r]$.

The DIVIDE step is cheap — takes only $\Theta(1)$ time.

But the COMBINE step is costly — takes $\Theta(n)$ time, where n is the length of $A[p..r]$.

The Divide-and-Conquer Process in Quicksort

Suppose we want to sort a typical subarray $A[p..r]$.

DIVIDE: Partition $A[p..r]$ into two (possibly empty) subarrays

$A[p..q - 1]$ and $A[q + 1..r]$ and find index q such that

- each element of $A[p..q - 1]$ is $\leq A[q]$, and
- each element of $A[q + 1..r]$ is $\geq A[q]$.

CONQUER: Recursively sort $A[p..q - 1]$ and $A[q + 1..r]$.

COMBINE: Since $A[q]$ is “equal or larger” and “equal or smaller” than everything to its left and right, respectively, and both left and right parts are sorted, subarray $A[p..r]$ is also sorted.

The COMBINE step is cheap — takes only $\Theta(1)$ time.

But the DIVIDE step is costly — takes $\Theta(n)$ time, where n is the length of $A[p..r]$.

Quicksort

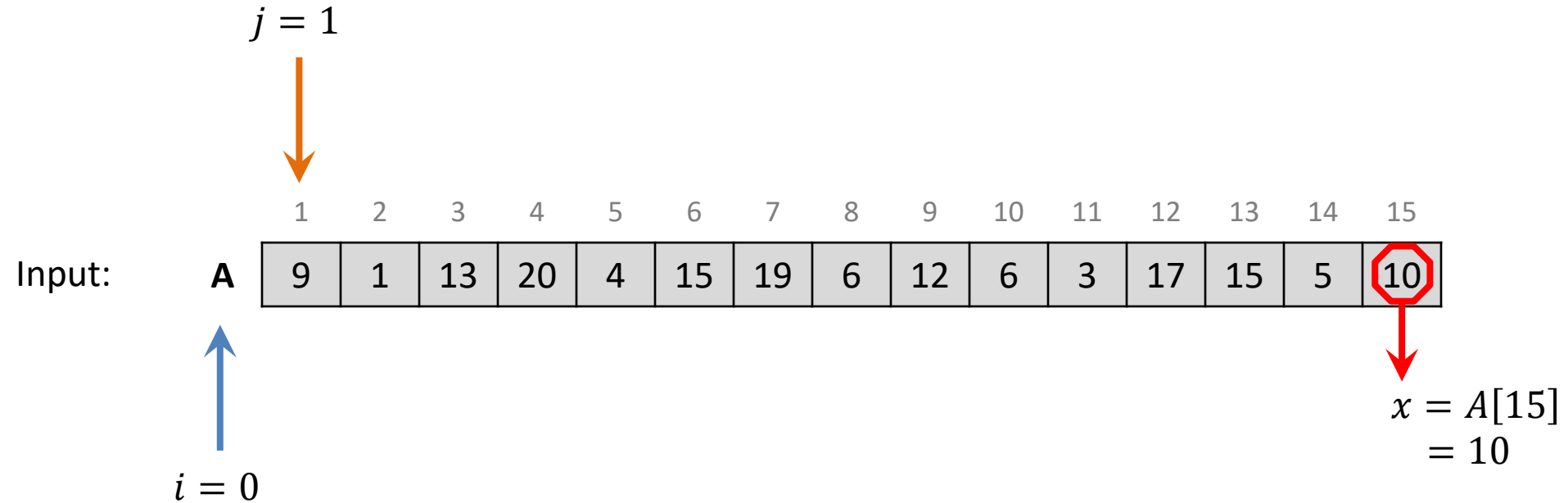
Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ rearranged in non-decreasing order of value.

QUICKSORT (A, p, r)

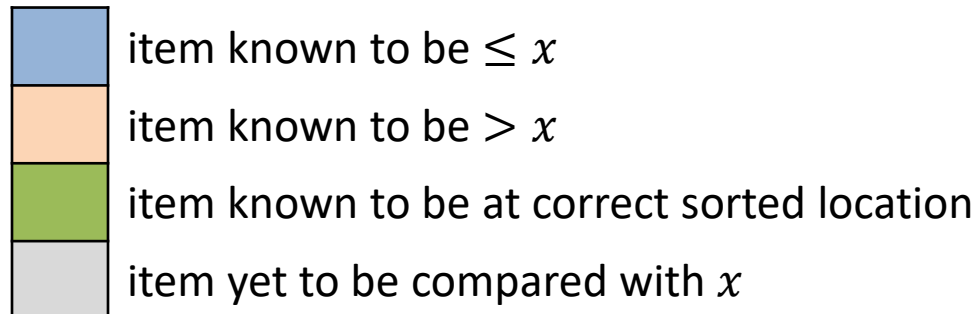
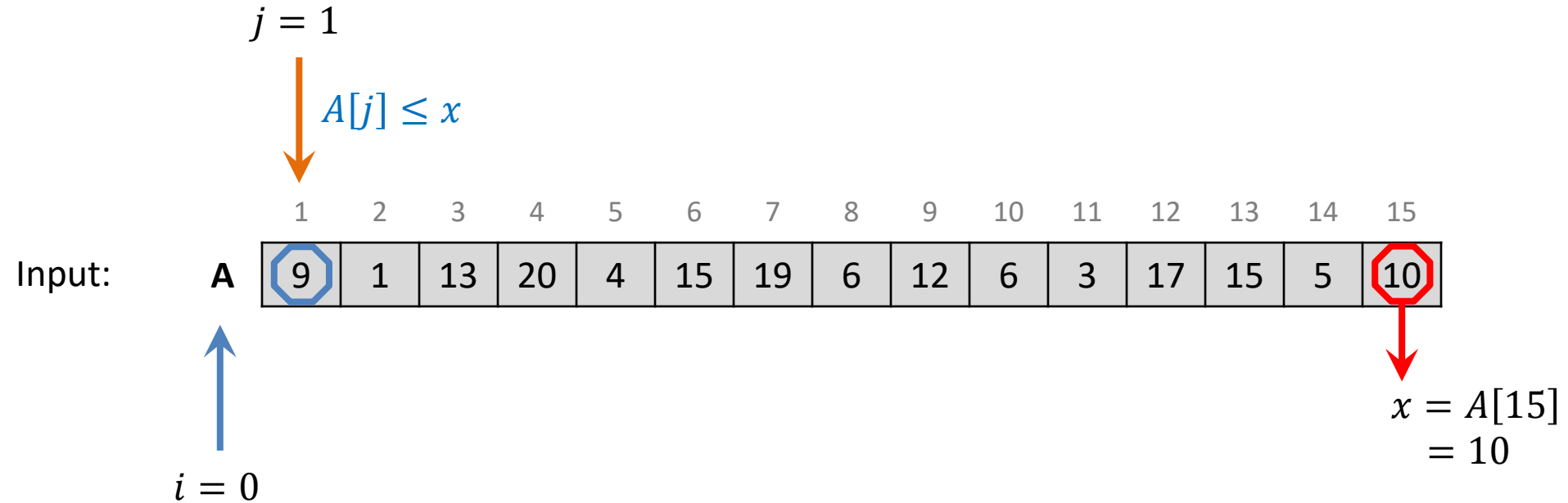
1. **if** $p < r$ **then**
2. // partition $A[p..r]$ into $A[p..q - 1]$ and $A[q + 1..r]$ such that everything in
 $A[p..q - 1]$ is $\leq A[q]$ and everything in $A[q + 1..r]$ is $\geq A[q]$
3. $q =$ PARTITION (A, p, r)
4. // recursively sort the left part
5. QUICKSORT ($A, p, q - 1$)
6. // recursively sort the right part
7. QUICKSORT ($A, q + 1, r$)

Partition

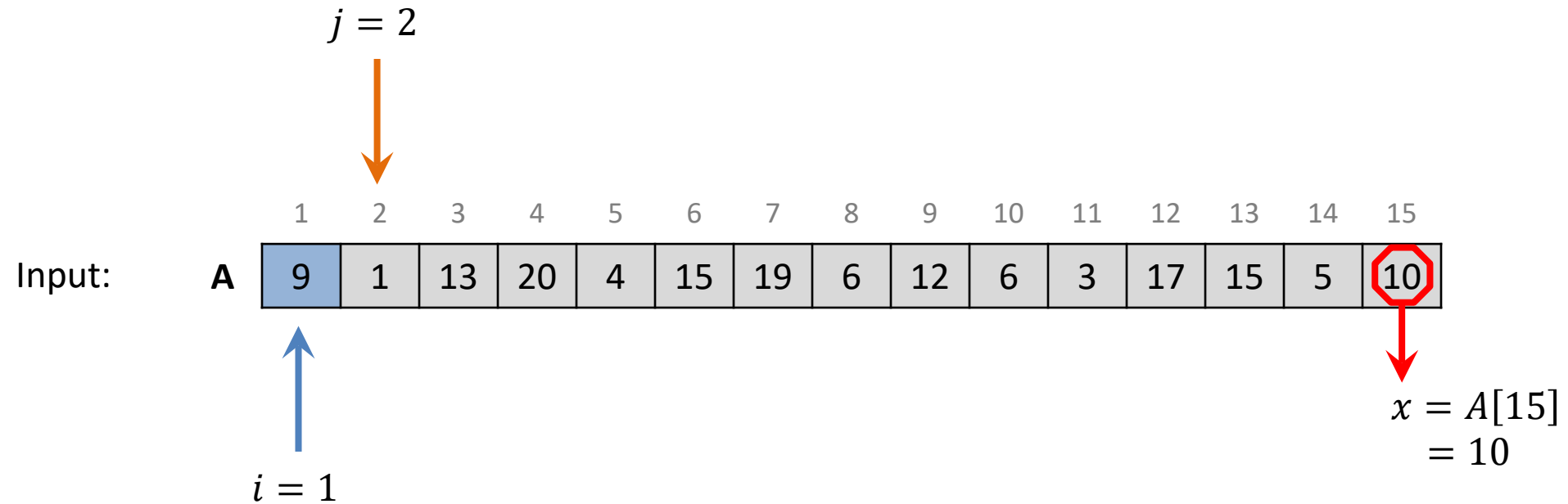


	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

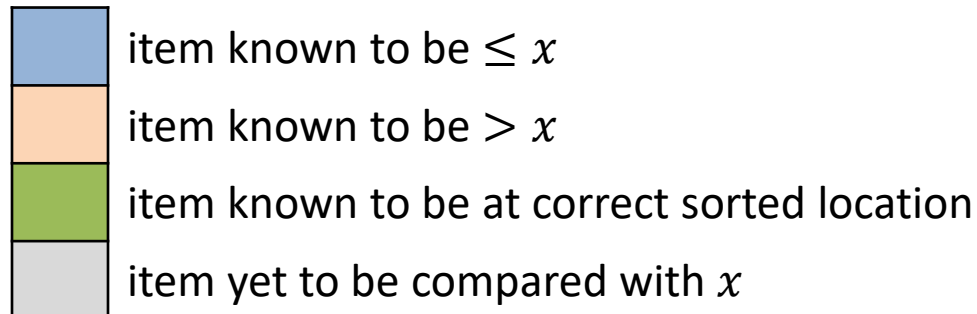
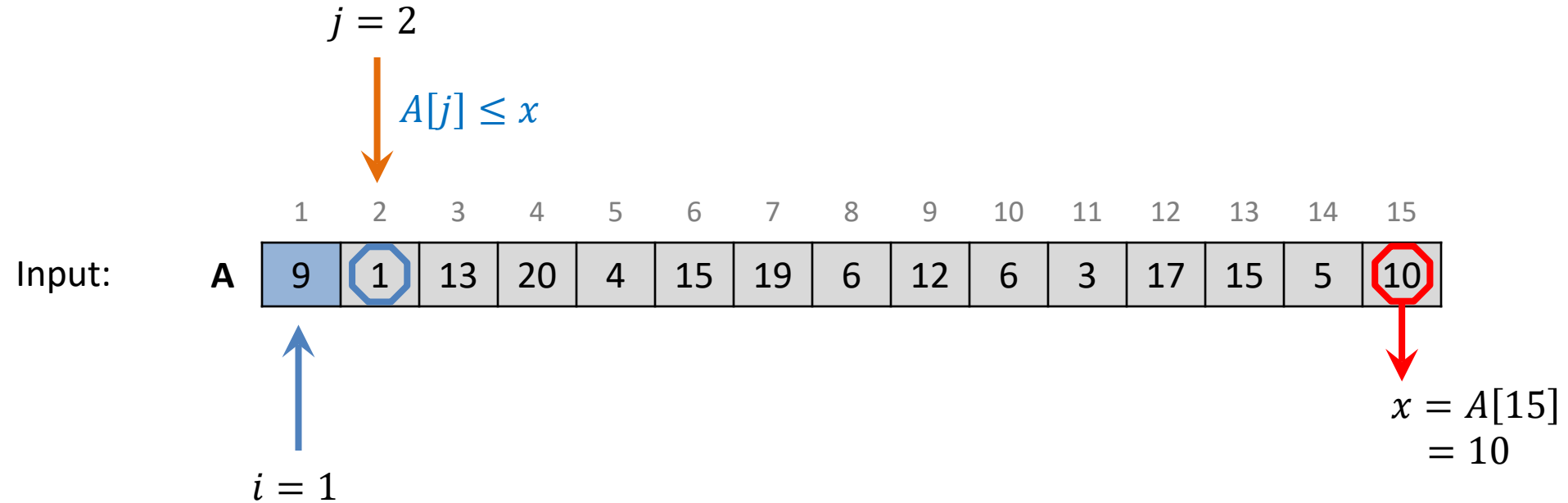


Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



Partition

$j = 3$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	13	20	4	15	19	6	12	6	3	17	15	5	10
---	---	----	----	---	----	----	---	----	---	---	----	----	---	----



$i = 2$

$x = A[15]$
 $= 10$



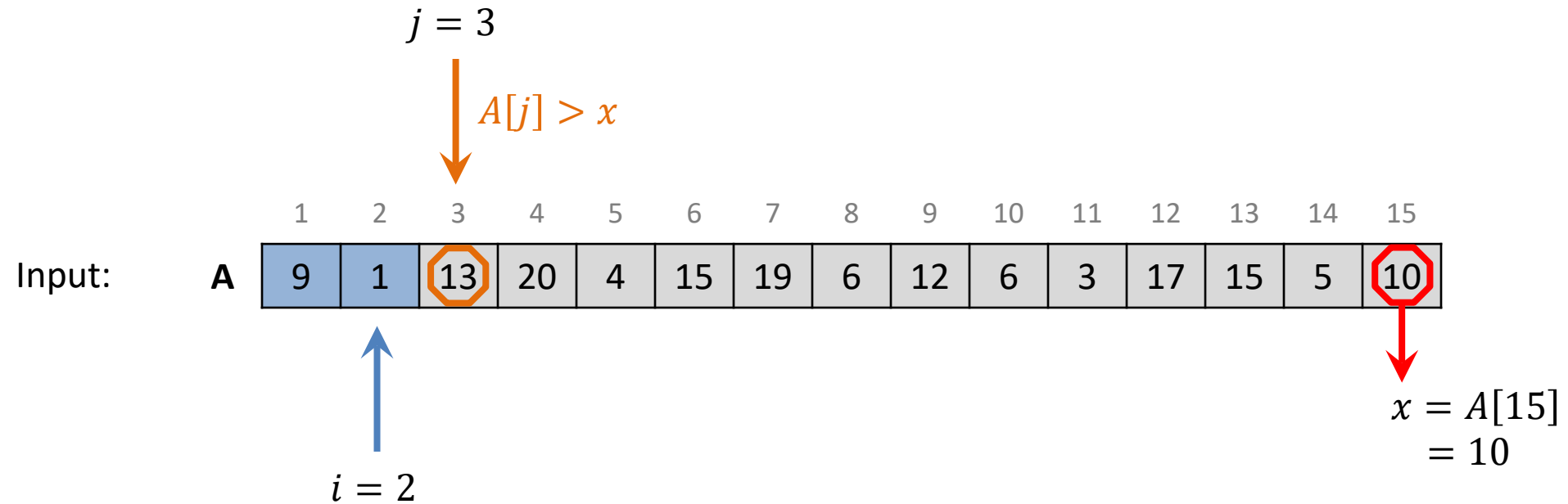
item known to be $\leq x$

item known to be $> x$

item known to be at correct sorted location

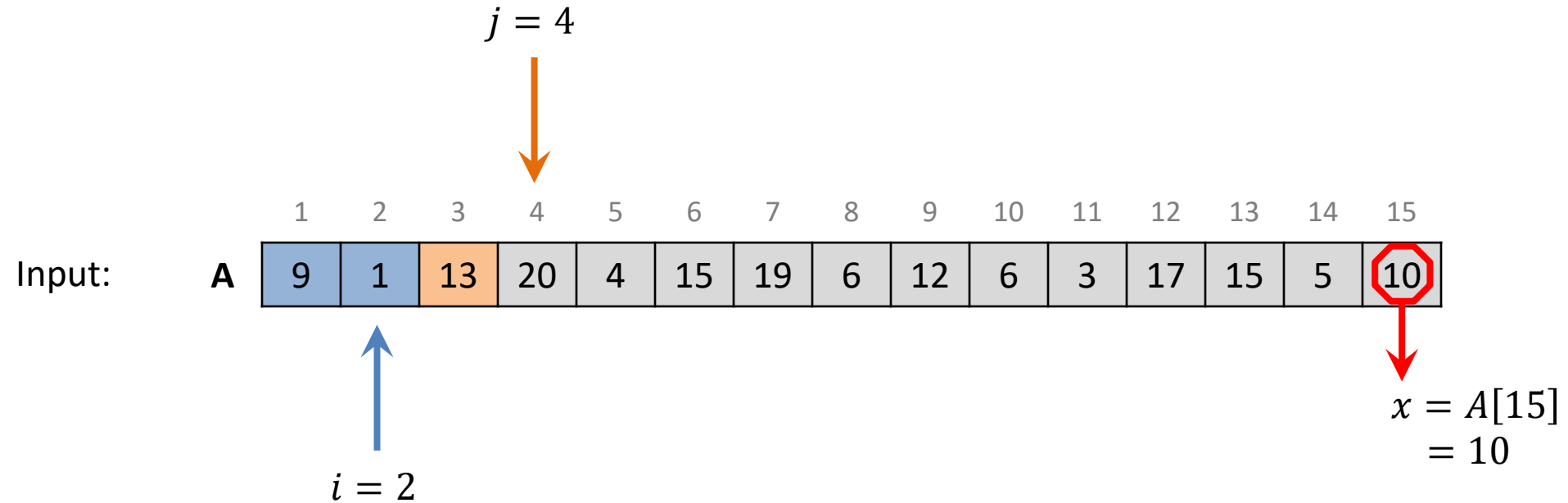
item yet to be compared with x

Partition



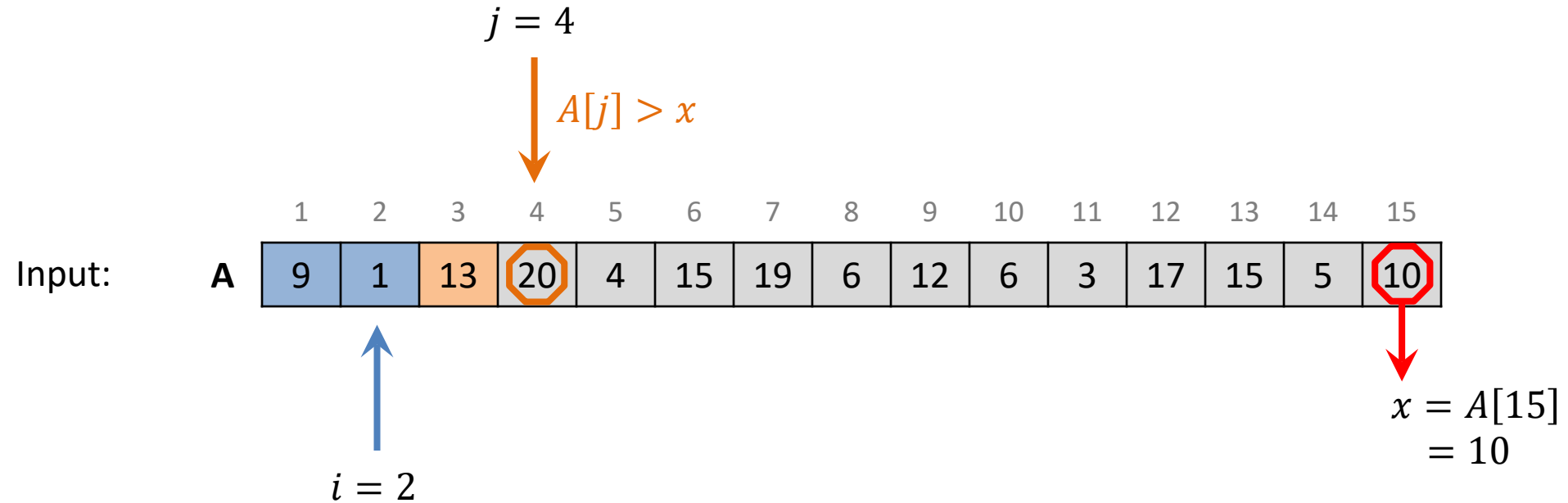
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



- item known to be $\leq x$
- item known to be $> x$
- item known to be at correct sorted location
- item yet to be compared with x

Partition

$j = 5$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	13	20	4	15	19	6	12	6	3	17	15	5	10
---	---	----	----	---	----	----	---	----	---	---	----	----	---	----



$i = 2$

$x = A[15]$
 $= 10$



item known to be $\leq x$

item known to be $> x$

item known to be at correct sorted location

item yet to be compared with x

Partition

$j = 5$

$A[j] \leq x$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	13	20	4	15	19	6	12	6	3	17	15	5	10
---	---	----	----	---	----	----	---	----	---	---	----	----	---	----



$i = 2$

$x = A[15]$
 $= 10$



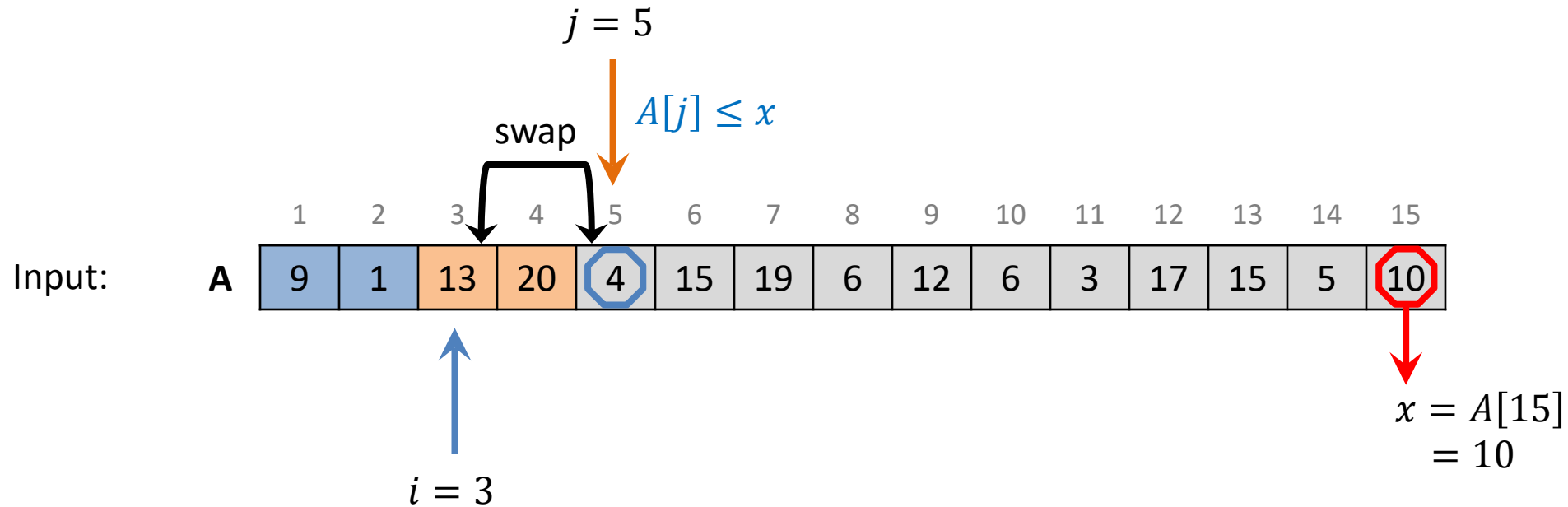
item known to be $\leq x$

item known to be $> x$

item known to be at correct sorted location

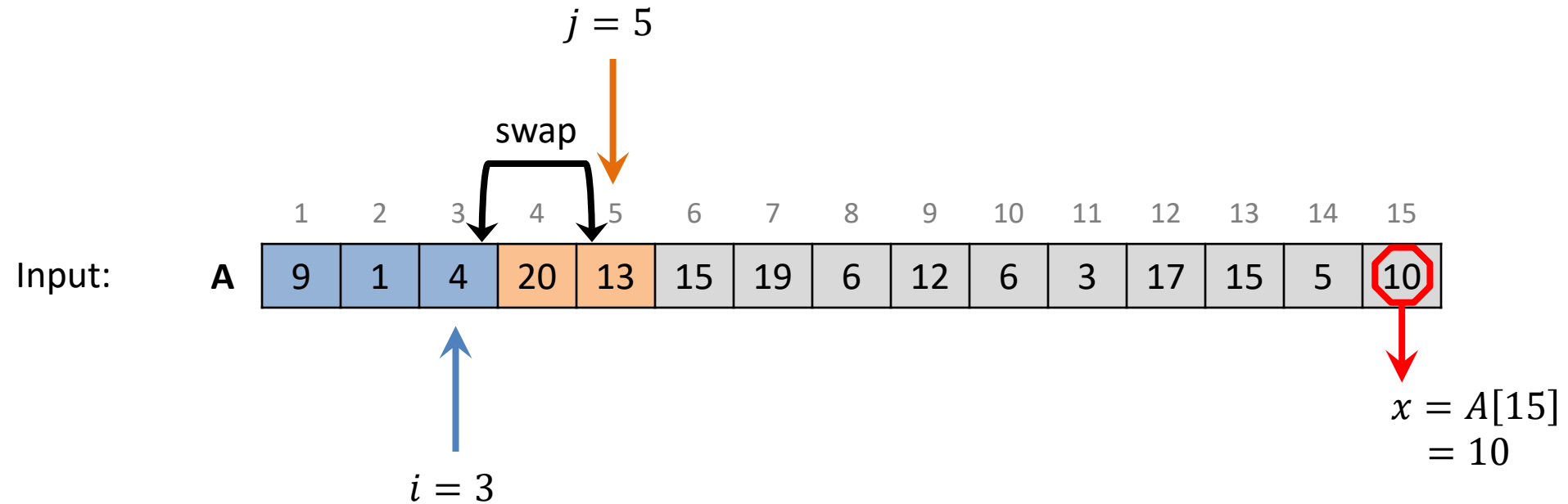
item yet to be compared with x

Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

$j = 6$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	4	20	13	15	19	6	12	6	3	17	15	5	10
---	---	---	----	----	----	----	---	----	---	---	----	----	---	----



$i = 3$

$x = A[15]$
 $= 10$

	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

$j = 6$

$A[j] > x$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	4	20	13	15	19	6	12	6	3	17	15	5	10
---	---	---	----	----	----	----	---	----	---	---	----	----	---	----



$i = 3$

$x = A[15]$
 $= 10$



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

$j = 7$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Input:

A

9	1	4	20	13	15	19	6	12	6	3	17	15	5	10
---	---	---	----	----	----	----	---	----	---	---	----	----	---	----



$i = 3$

$x = A[15]$
 $= 10$

	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

$$j = 7$$

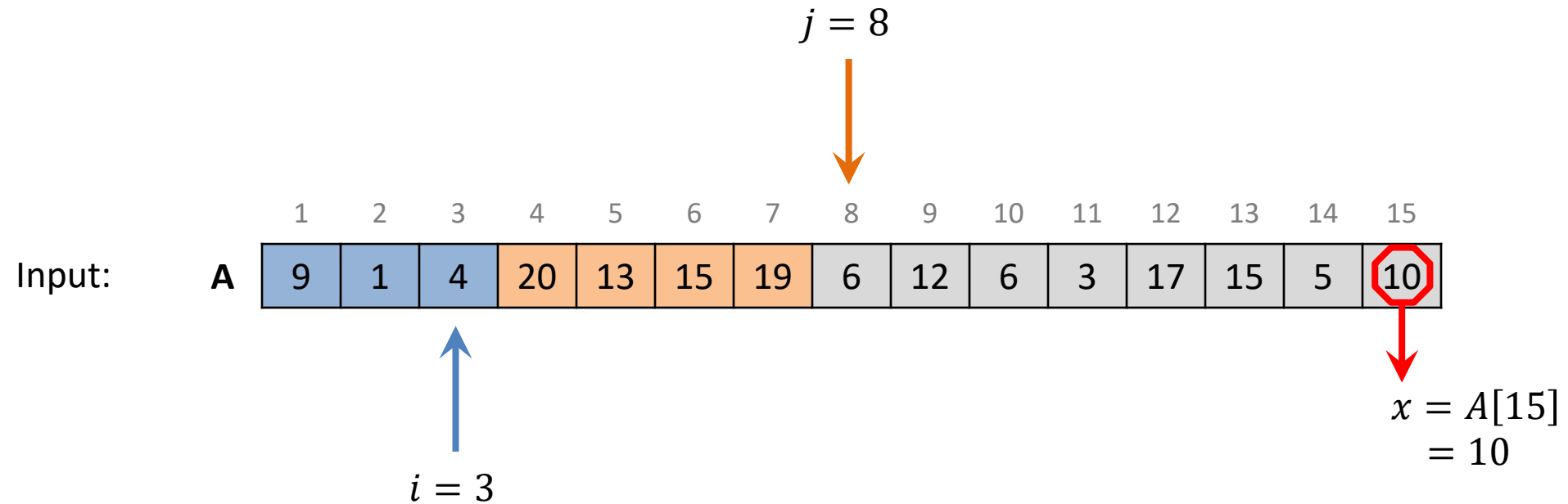
$$A[j] > x$$

Input:



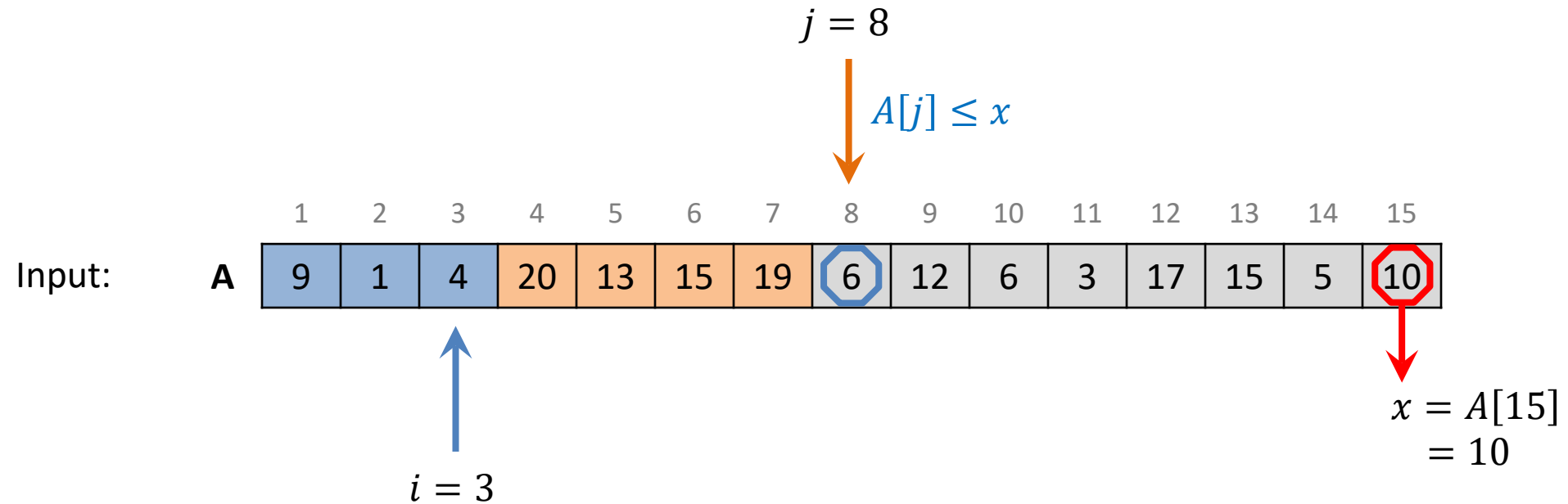
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



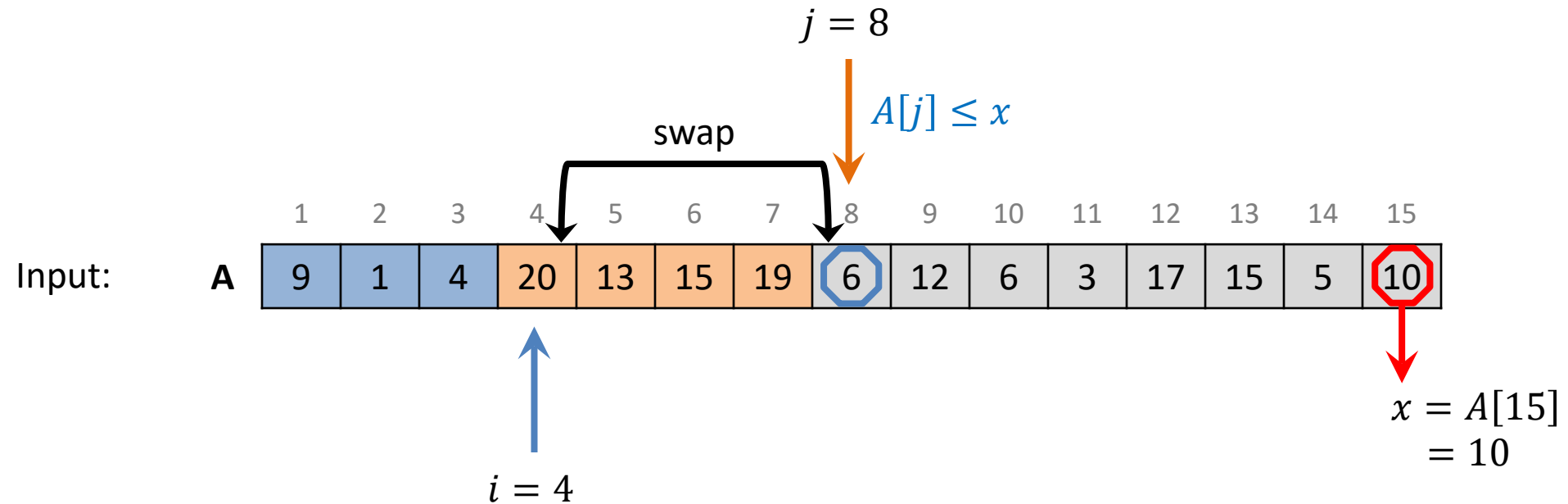
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



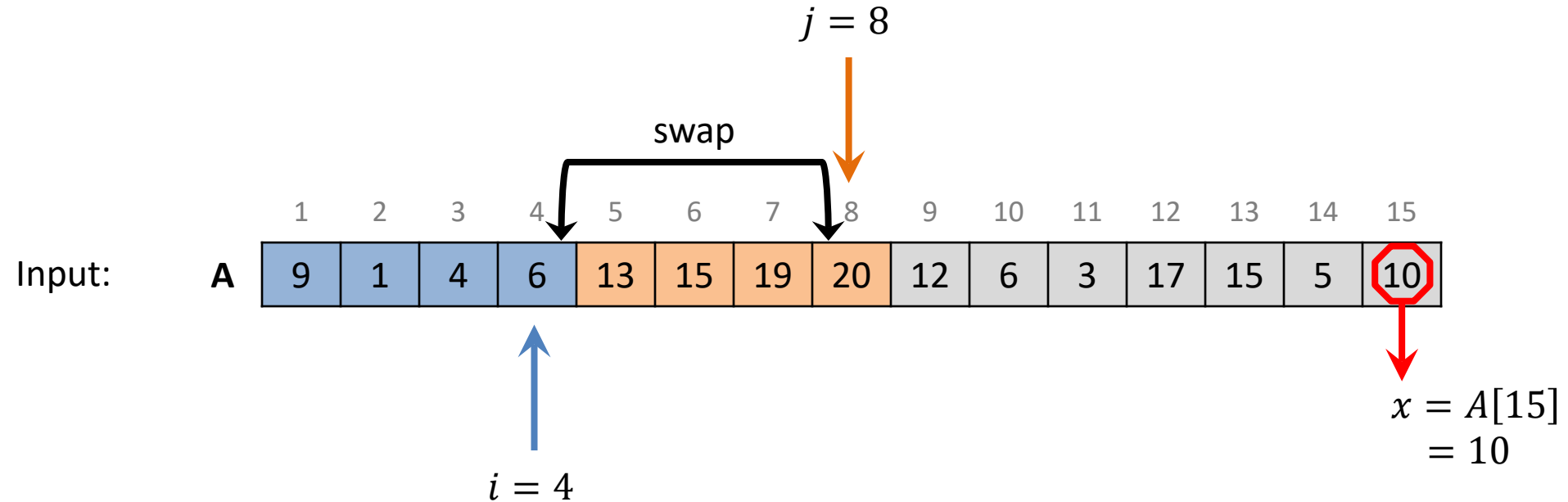
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



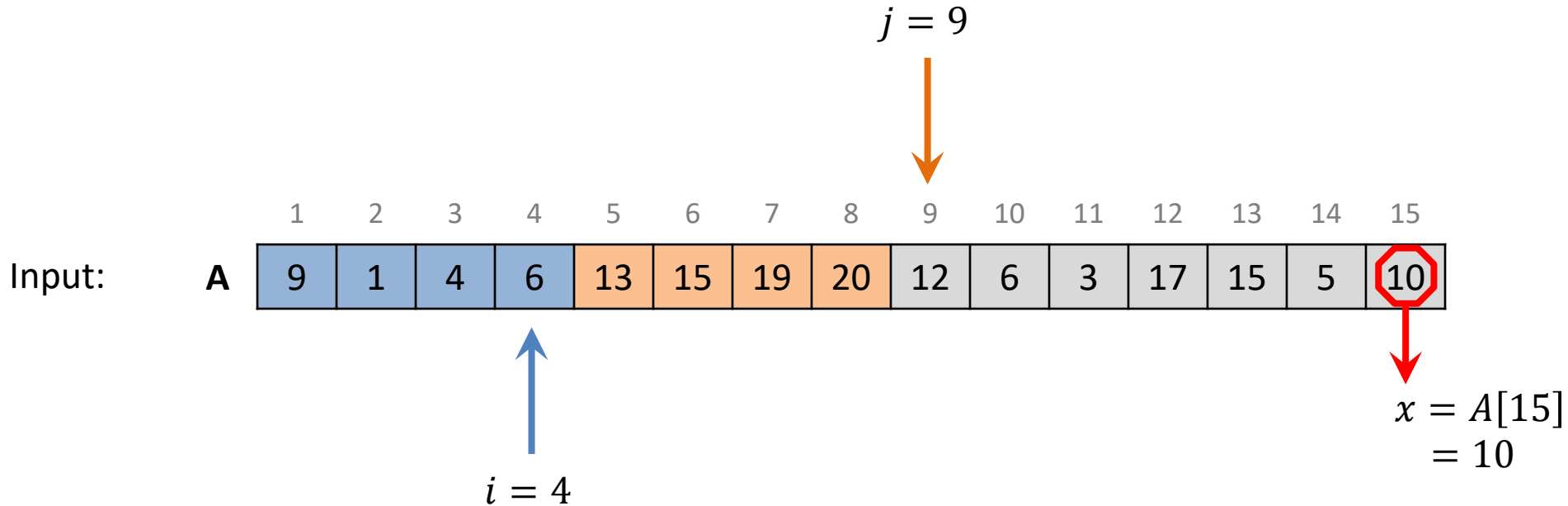
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



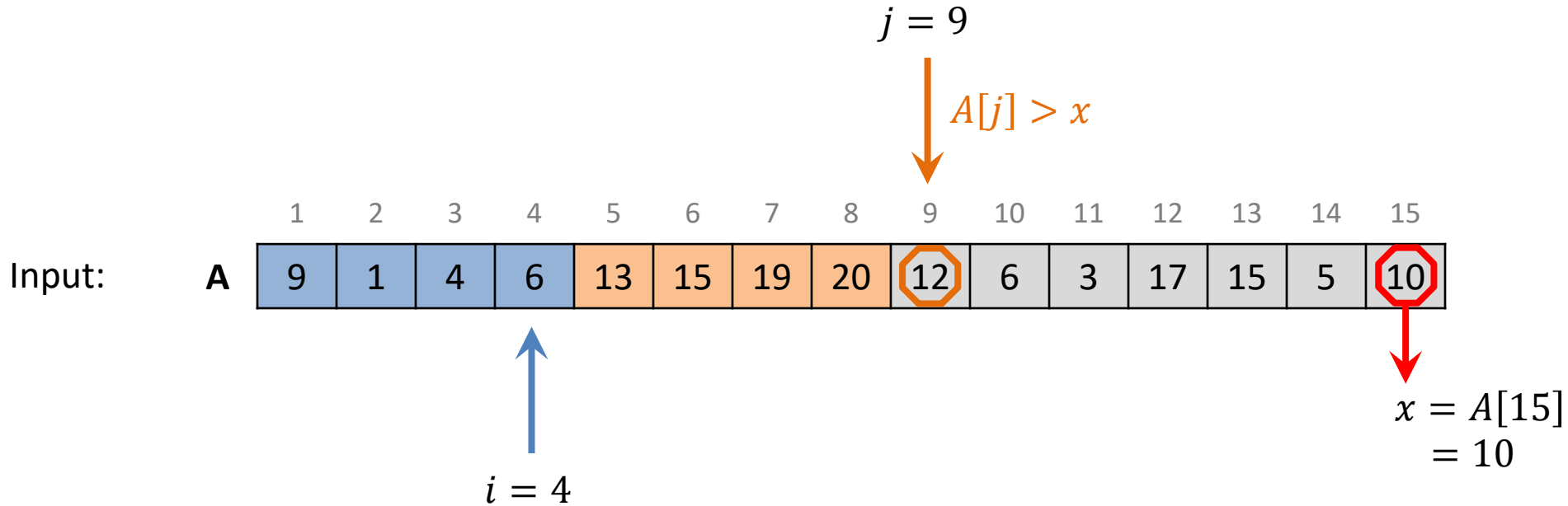
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



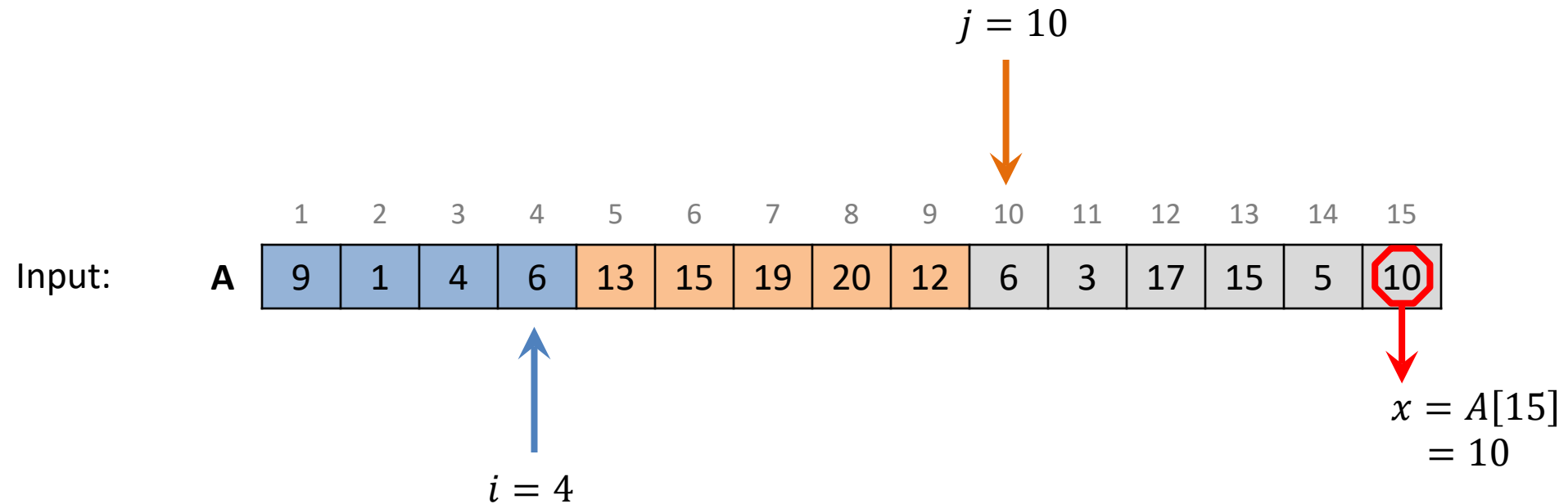
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



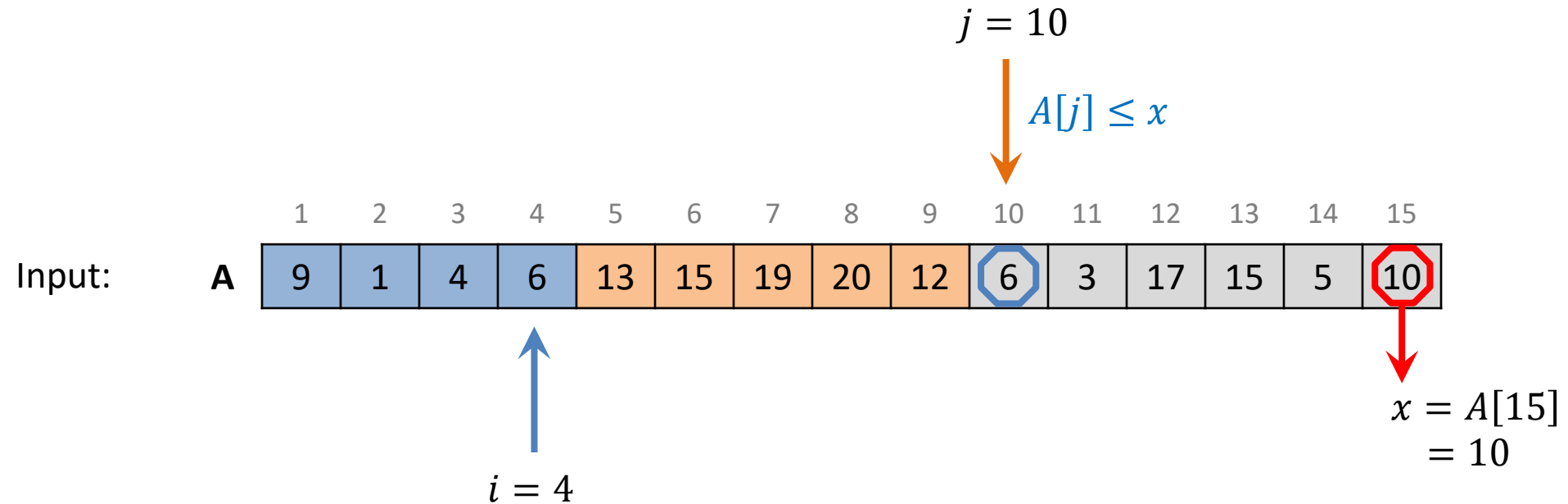
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



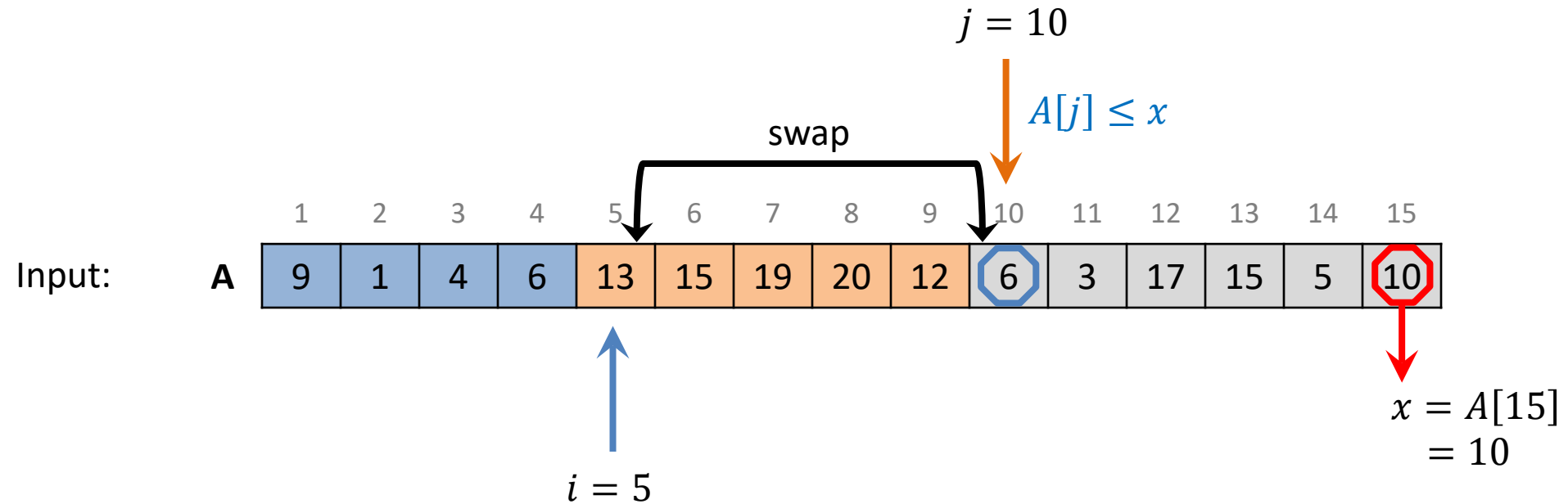
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



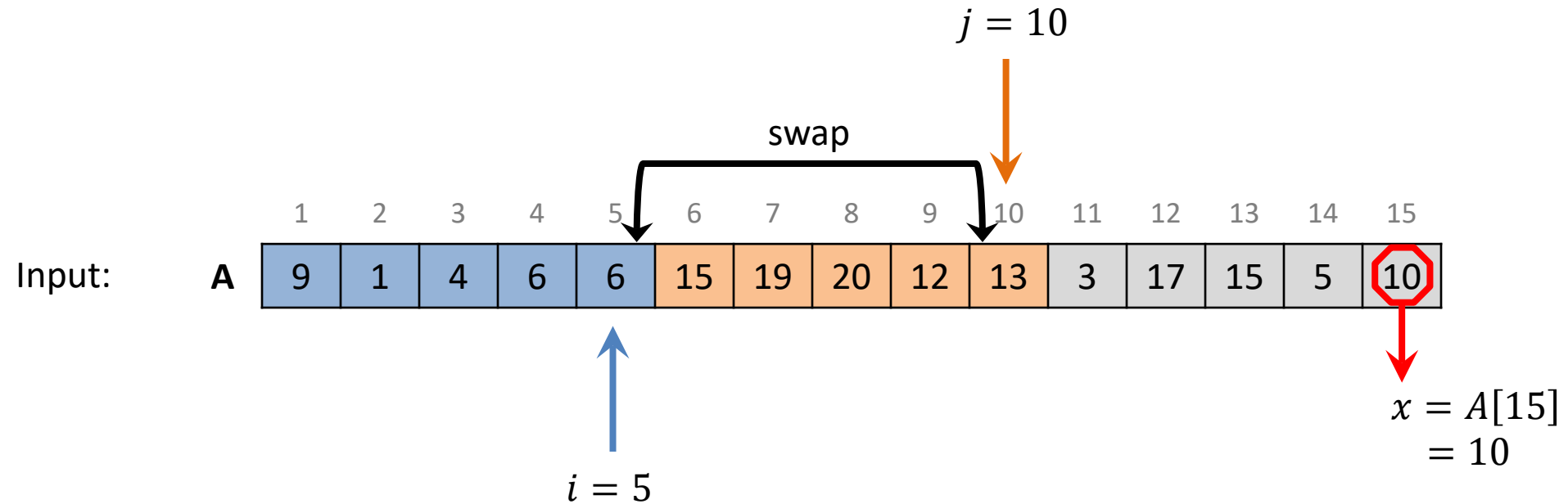
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



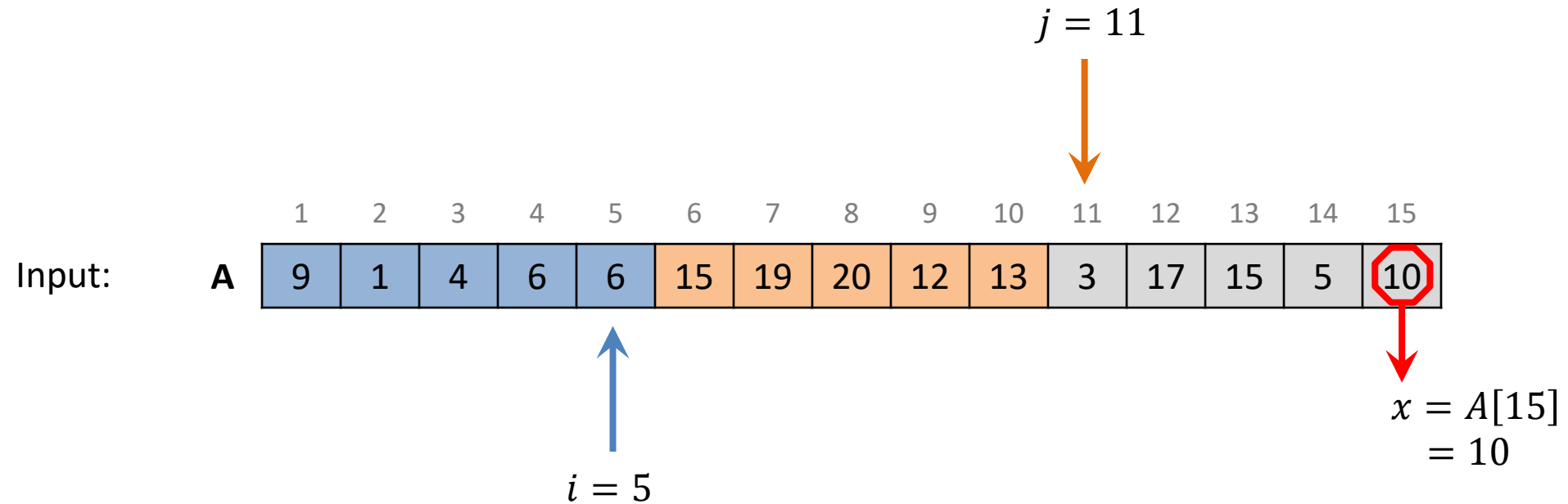
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



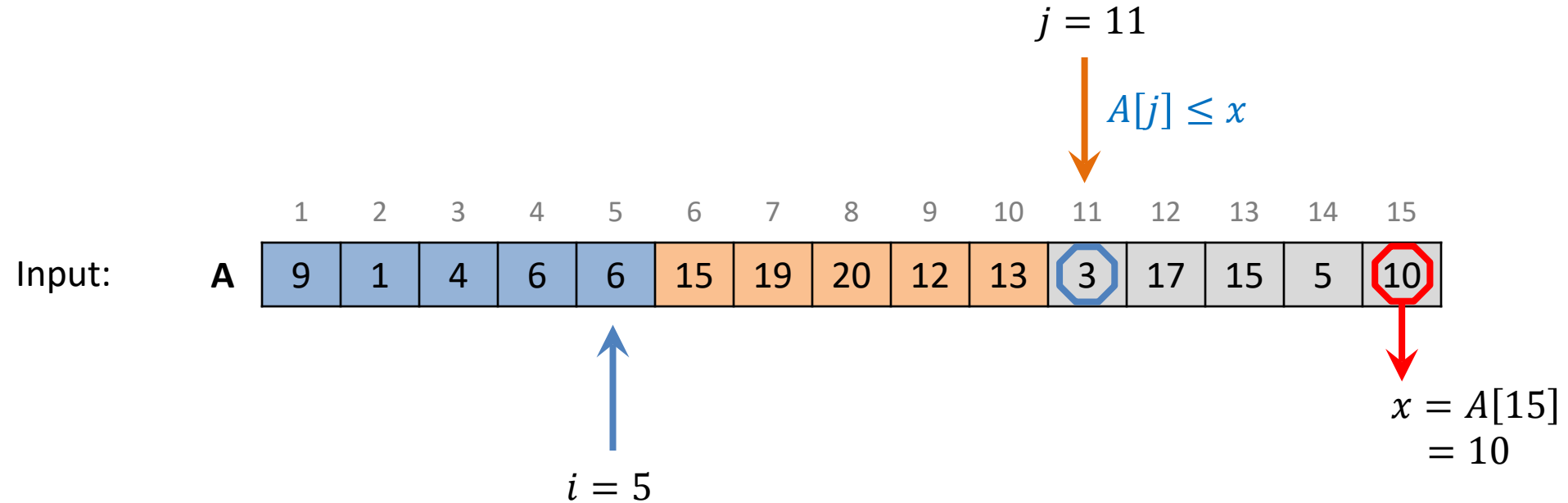
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



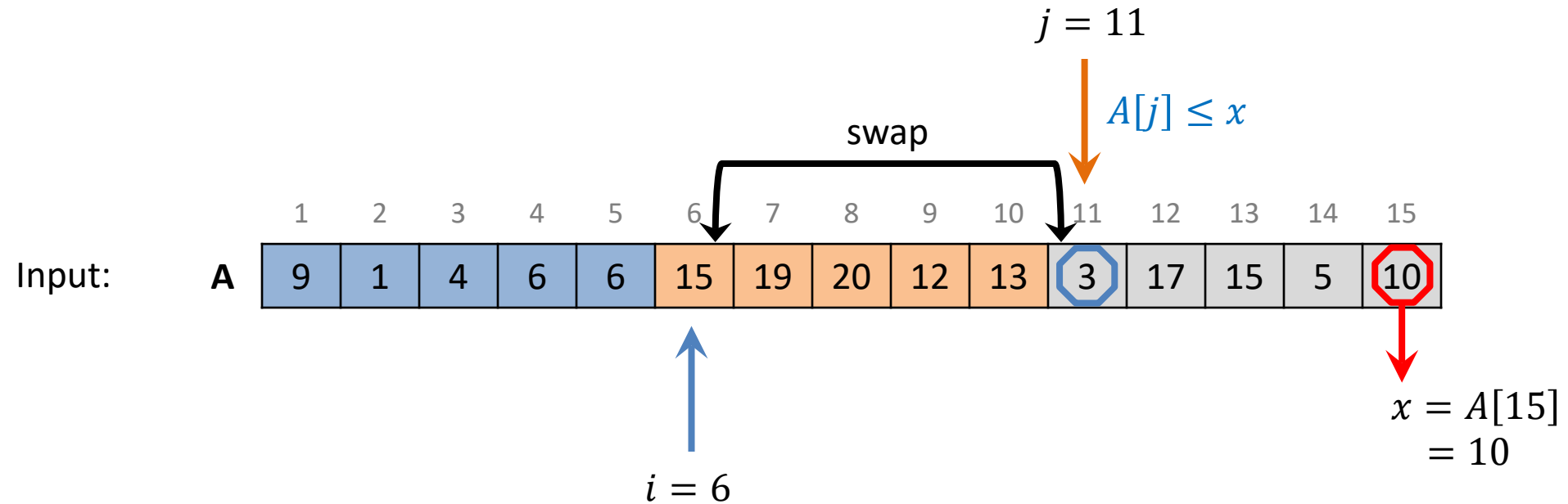
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



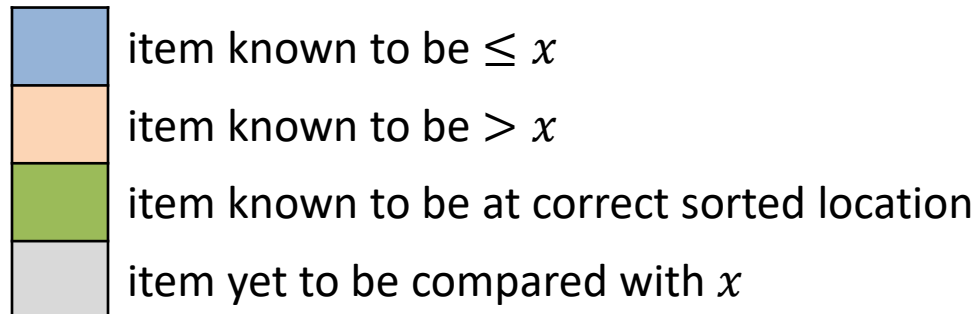
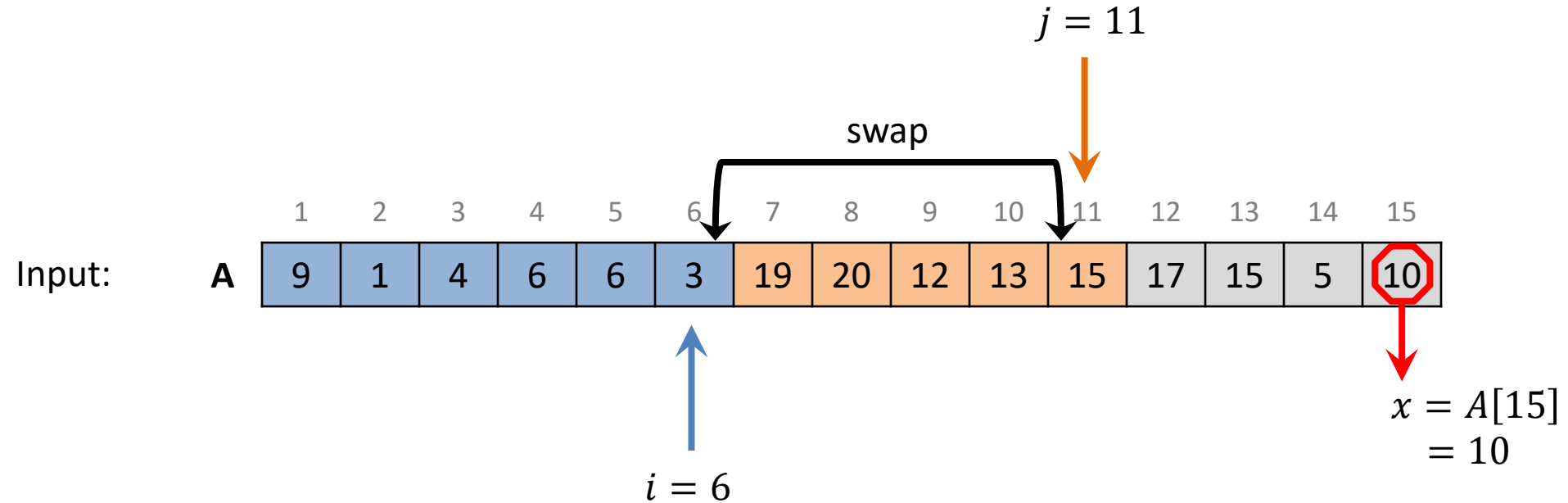
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

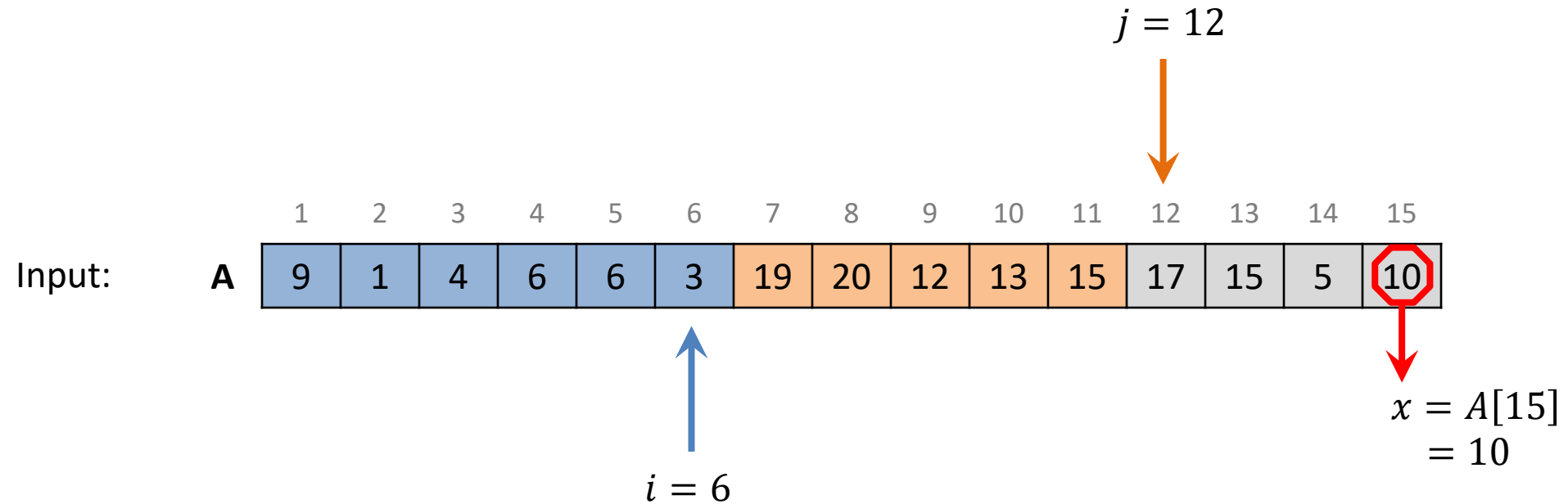


	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

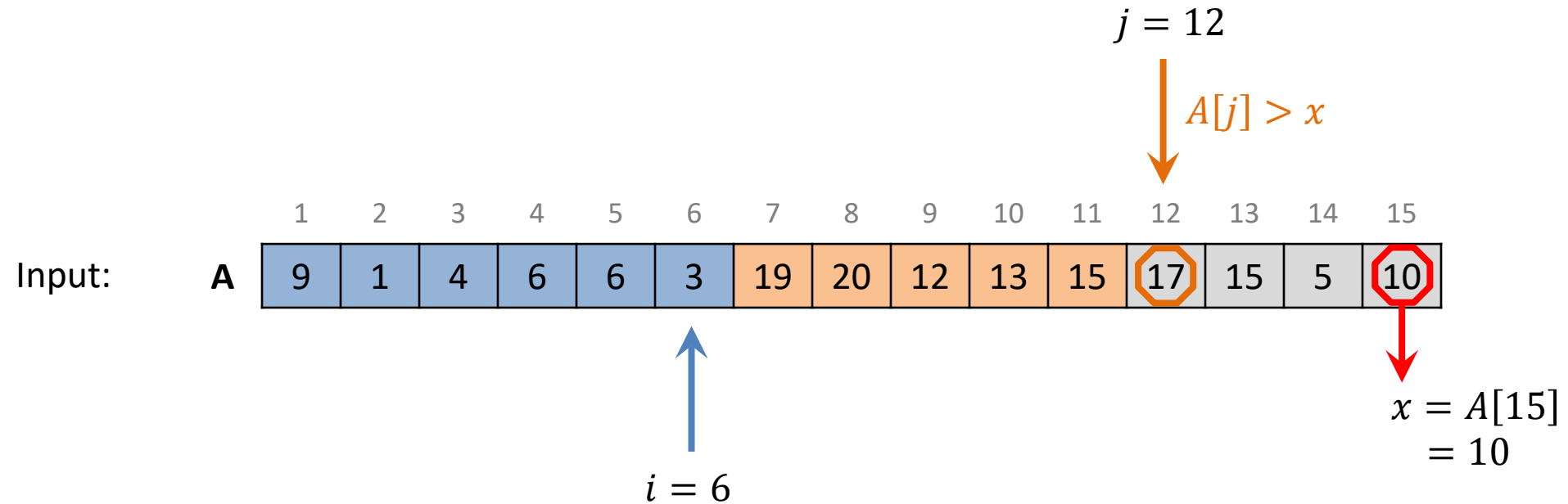


Partition



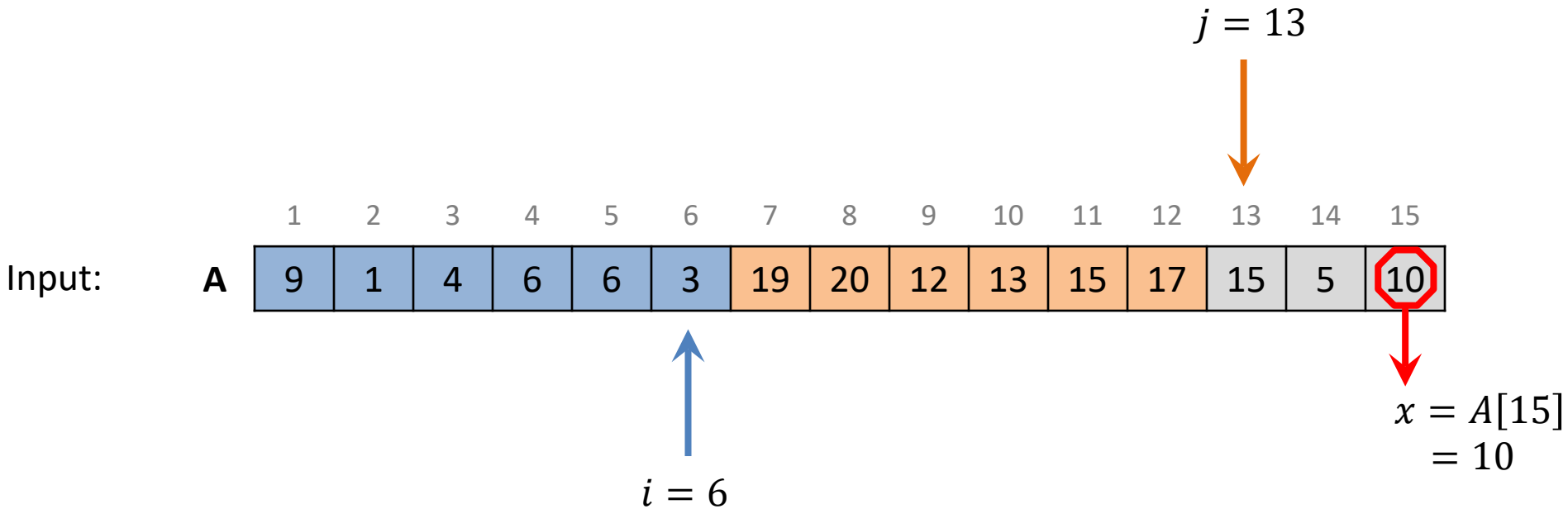
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



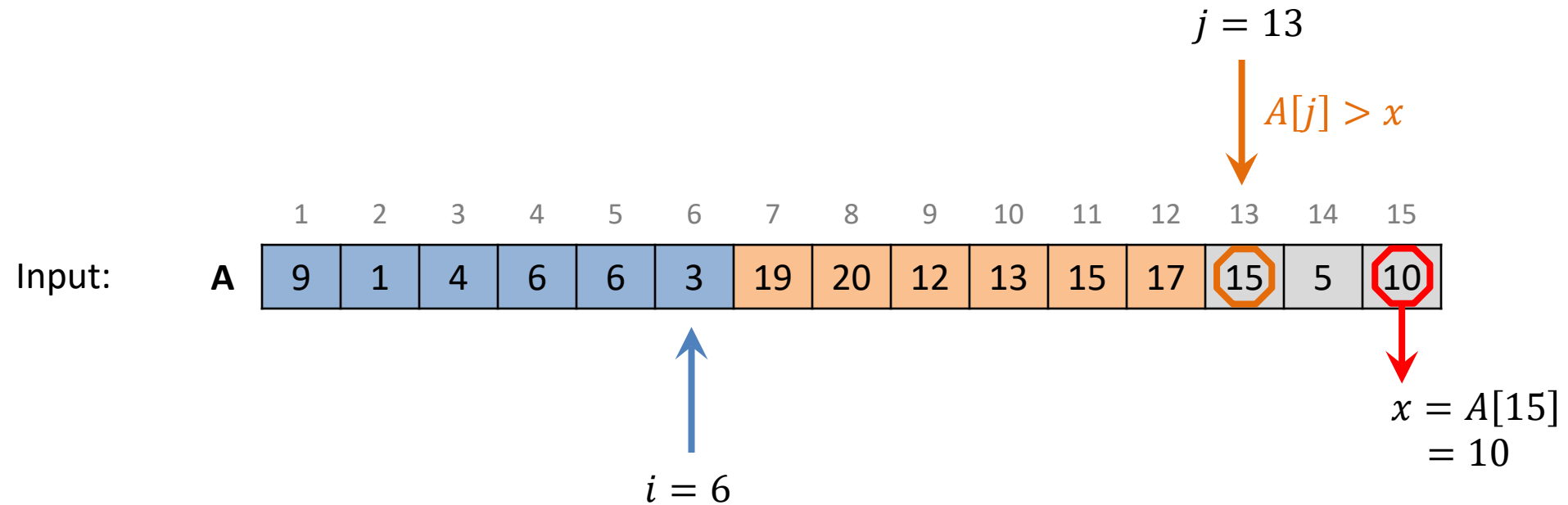
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



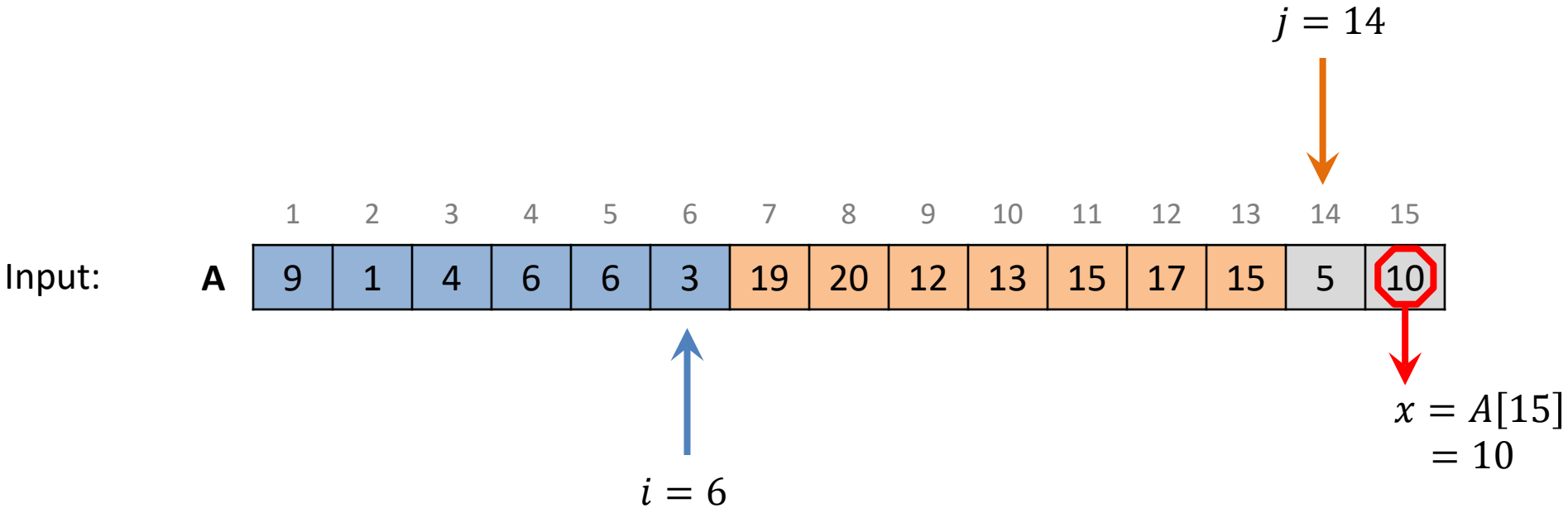
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



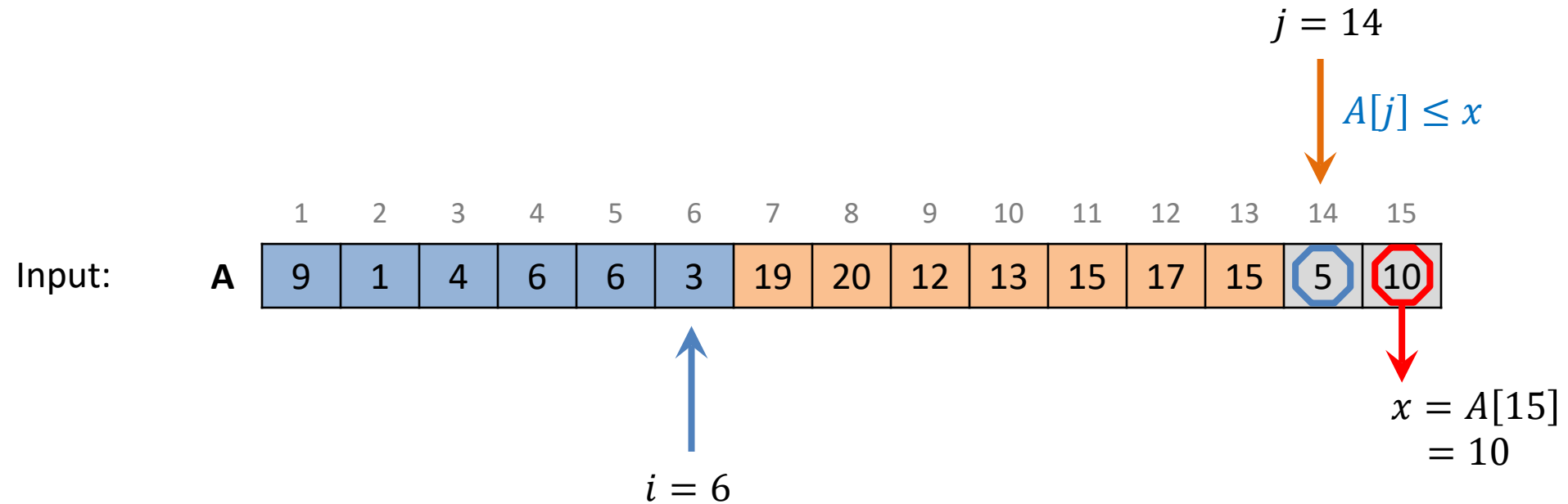
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



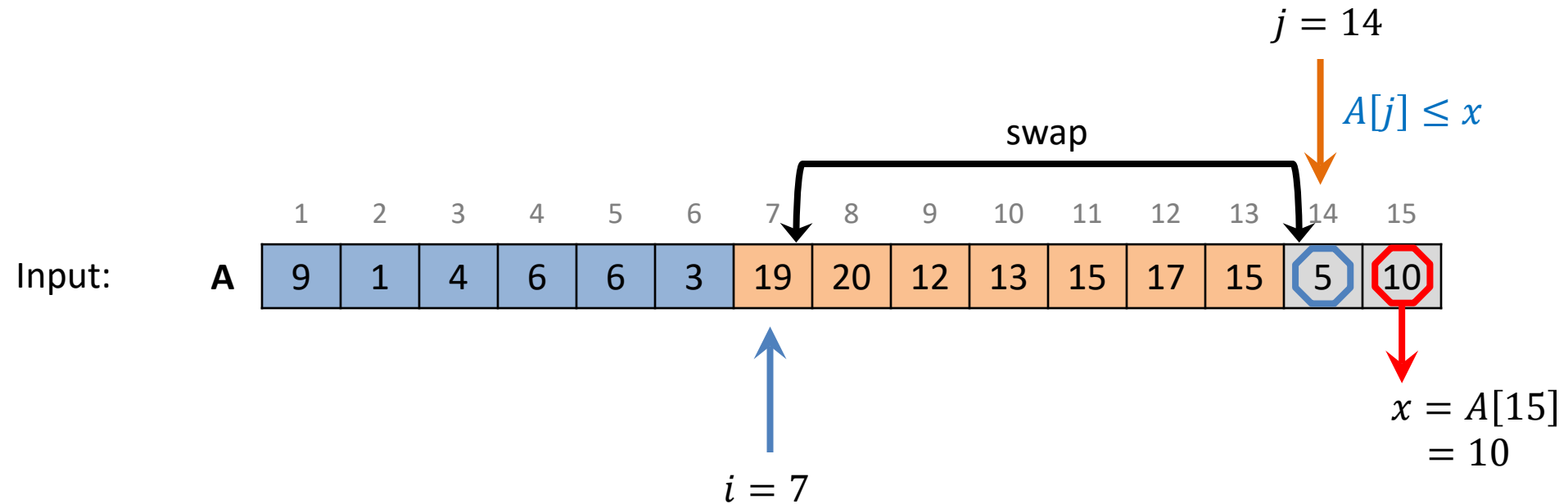
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



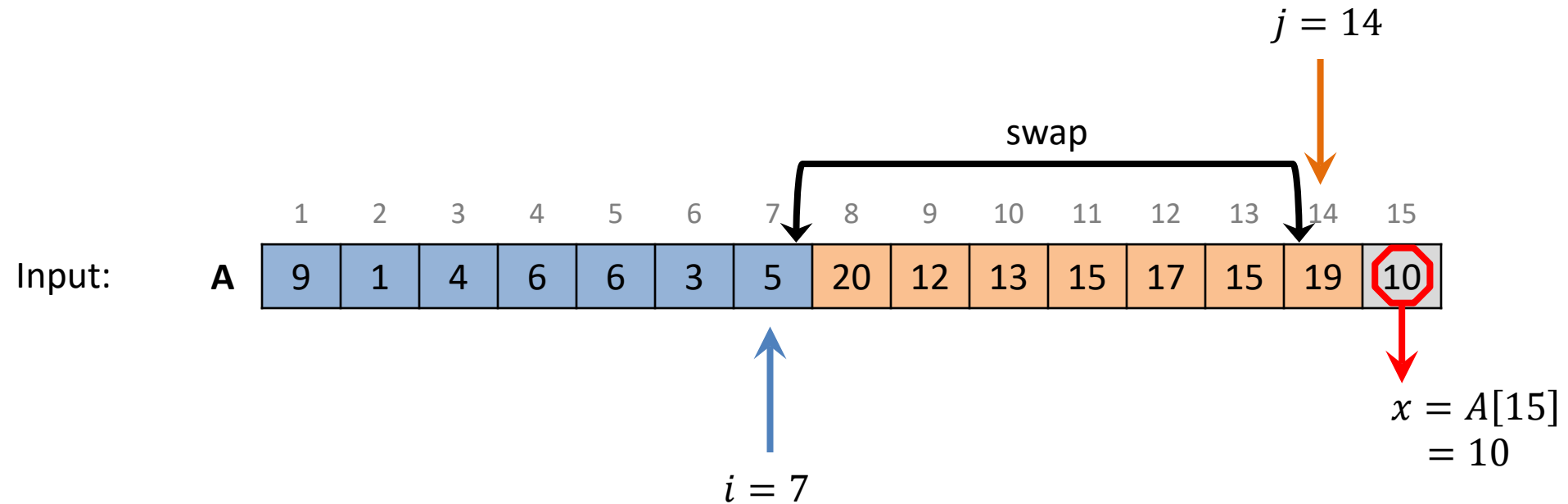
	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



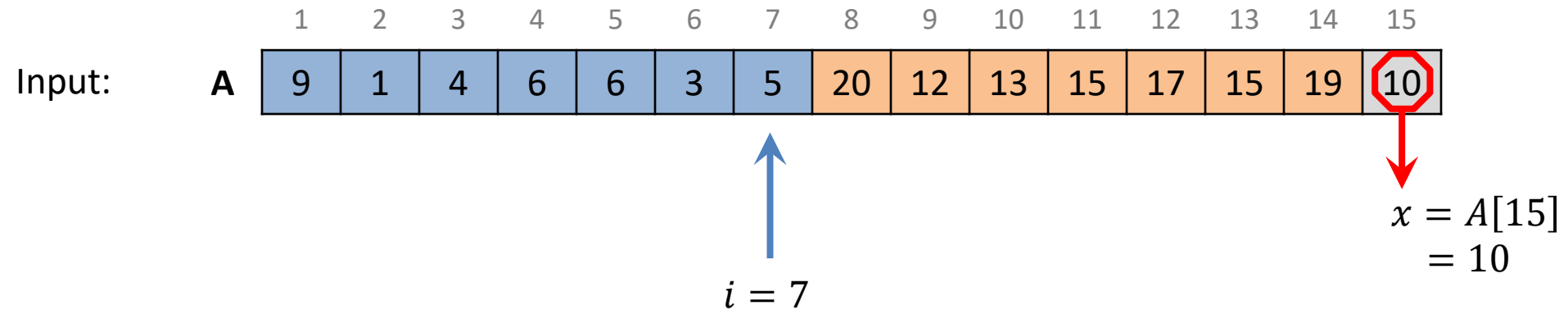
- item known to be $\leq x$
- item known to be $> x$
- item known to be at correct sorted location
- item yet to be compared with x

Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

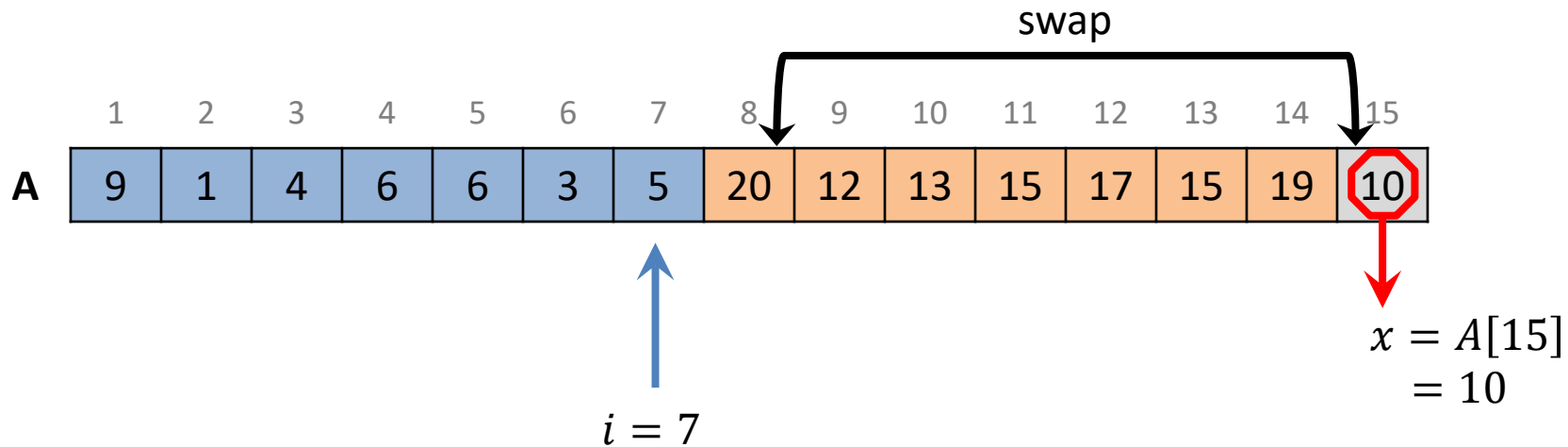
Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

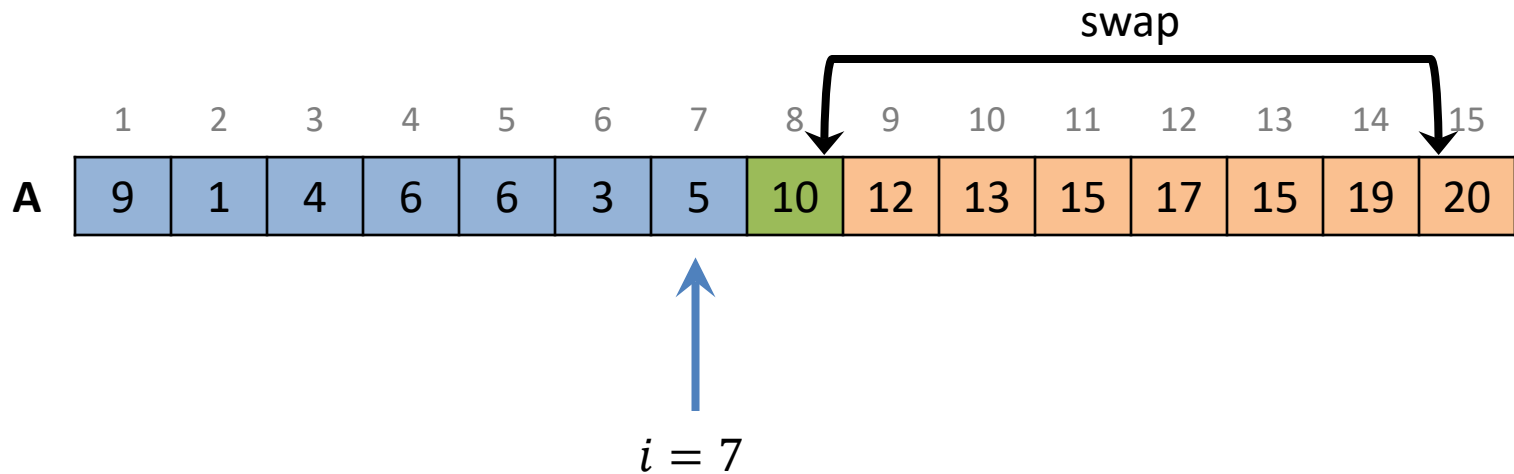
Input:



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

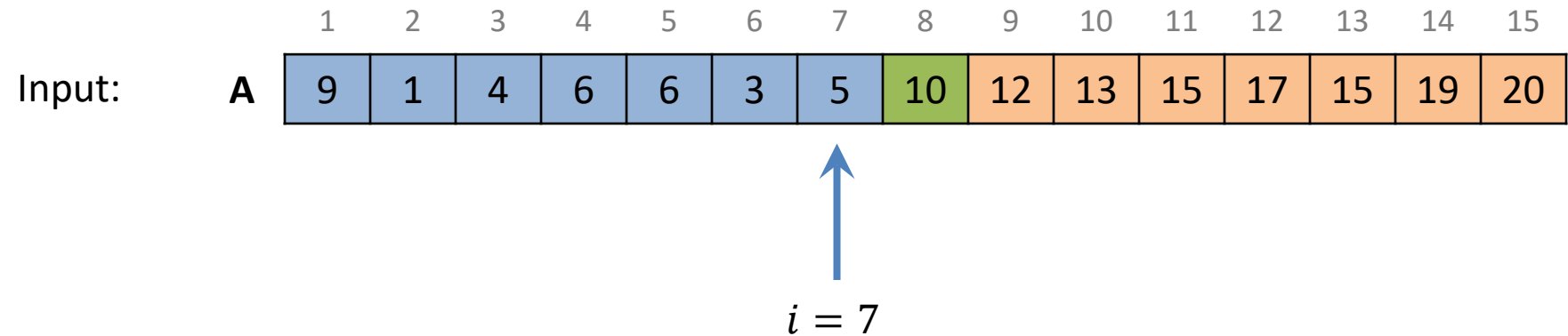
Partition

Input:



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition



	item known to be $\leq x$
	item known to be $> x$
	item known to be at correct sorted location
	item yet to be compared with x

Partition

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ are rearranged such that for some $q \in [p, r]$ everything in $A[p : q - 1]$ is $\leq A[q]$ and everything in $A[q + 1 : r]$ is $\geq A[q]$. Index q is returned.

PARTITION (A, p, r)

1. $x = A[r]$
2. $i = p - 1$
3. **for** $j = p$ **to** $r - 1$
4. **if** $A[j] \leq x$
5. $i = i + 1$
6. exchange $A[i]$ with $A[j]$
7. exchange $A[i + 1]$ with $A[r]$
8. **return** $i + 1$

Running Time of Partition

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ are rearranged such that for some $q \in [p, r]$ everything in $A[p : q - 1]$ is $\leq A[q]$ and everything in $A[q + 1 : r]$ is $\geq A[q]$. Index q is returned.

PARTITION (A, p, r)

1. $x = A[r]$
2. $i = p - 1$
3. **for** $j = p$ **to** $r - 1$
4. **if** $A[j] \leq x$
5. $i = i + 1$
6. exchange $A[i]$ with $A[j]$
7. exchange $A[i + 1]$ with $A[r]$
8. **return** $i + 1$

Let $n = r - p + 1$.

The loop of lines 3–6 takes $\Theta(r - 1 - p + 1) = \Theta(n)$ time.

Lines 1, 2, 7 and 8 take $\Theta(1)$ time each.

Hence, the overall running time is $\Theta(n)$.

Running Time of Partition

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ are rearranged such that for some $q \in [p, r]$ everything in $A[p : q - 1]$ is $\leq A[q]$ and everything in $A[q + 1 : r]$ is $\geq A[q]$. Index q is returned.

PARTITION (A, p, r)

1. $x = A[r]$
2. $i = p - 1$
3. **for** $j = p$ **to** $r - 1$
4. **if** $A[j] \leq x$
5. $i = i + 1$
6. exchange $A[i]$ with $A[j]$
7. exchange $A[i + 1]$ with $A[r]$
8. **return** $i + 1$

Let $n = r - p + 1$.

The loop of lines 3–6 takes $\Theta(r - 1 - p + 1) = \Theta(n)$ time.

Lines 1, 2, 7 and 8 take $\Theta(1)$ time each.

Hence, the overall running time is $\Theta(n)$.

Quicksort

Input:

A	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



item determined to be at correct sorted location

Quicksort

Partition $A[1..15]$ around 10

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	13	20	4	15	19	6	12	6	3	17	15	5	10



item determined to be at correct sorted location

Quicksort

Partition $A[1..15]$ around 10

Input:

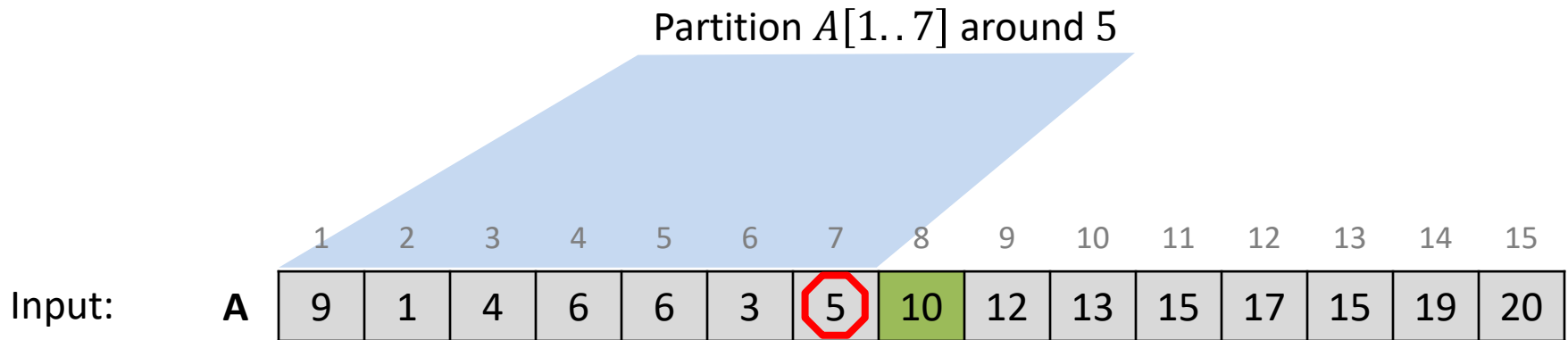
A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
9	1	4	6	6	3	5	10	12	13	15	17	15	19	20



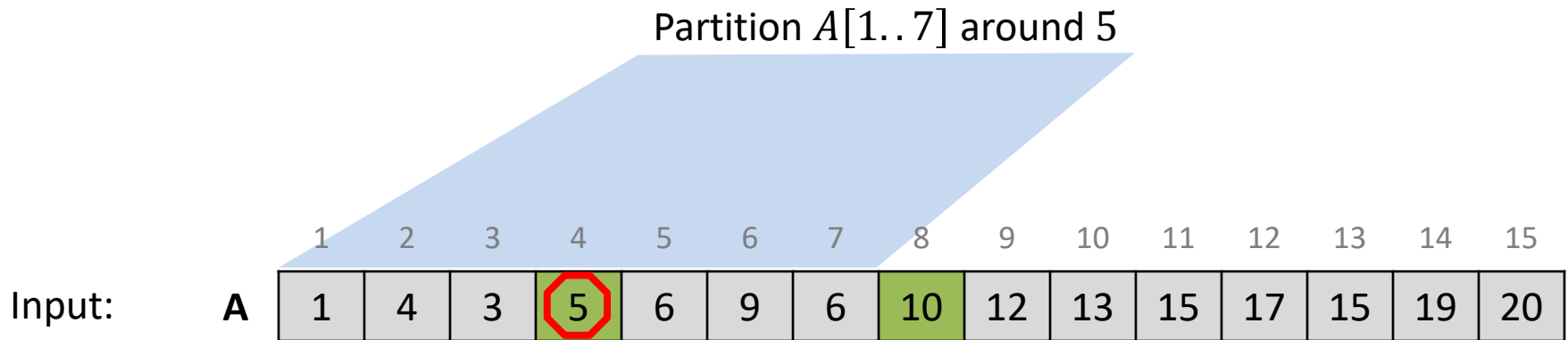
item determined to be at correct sorted location

Quicksort



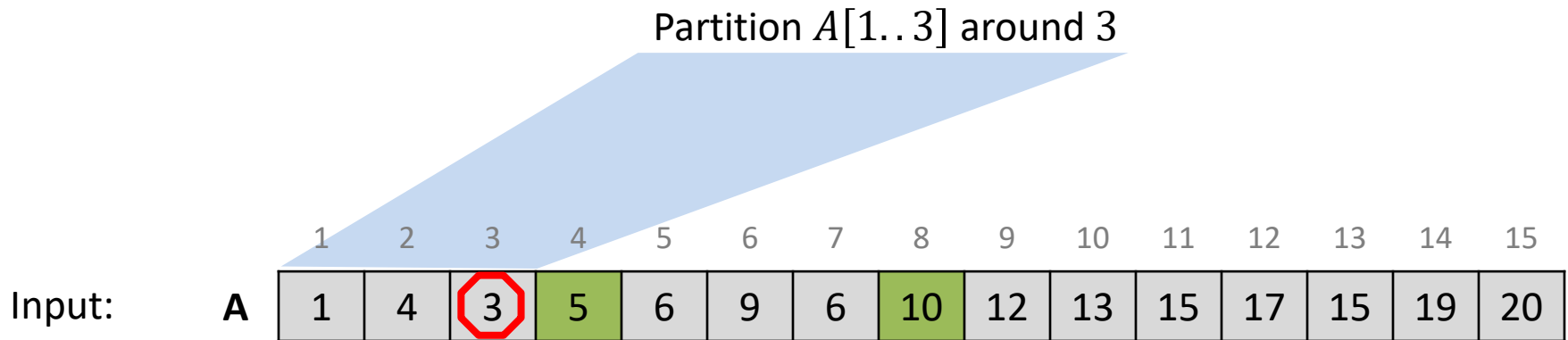
item determined to be at correct sorted location

Quicksort



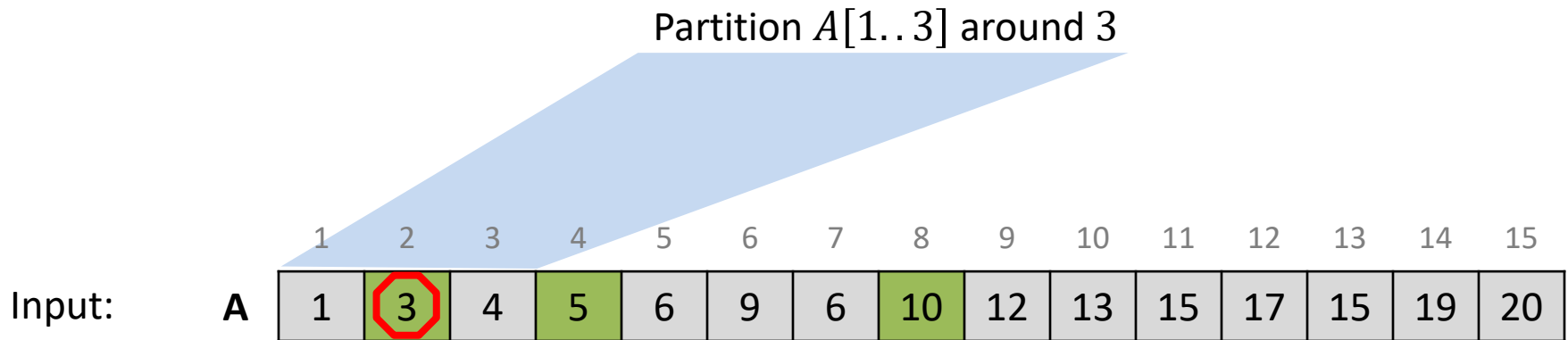
item determined to be at correct sorted location

Quicksort



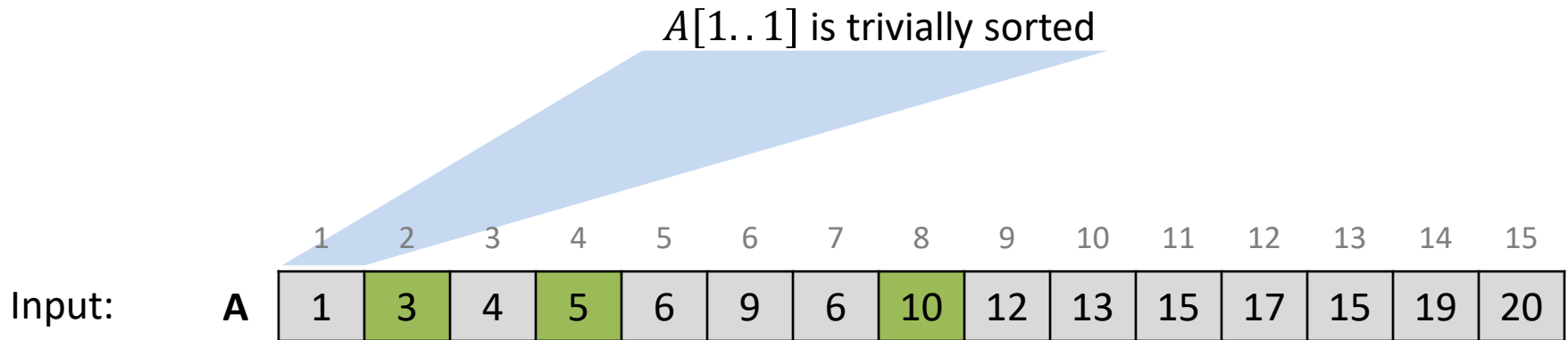
item determined to be at correct sorted location

Quicksort



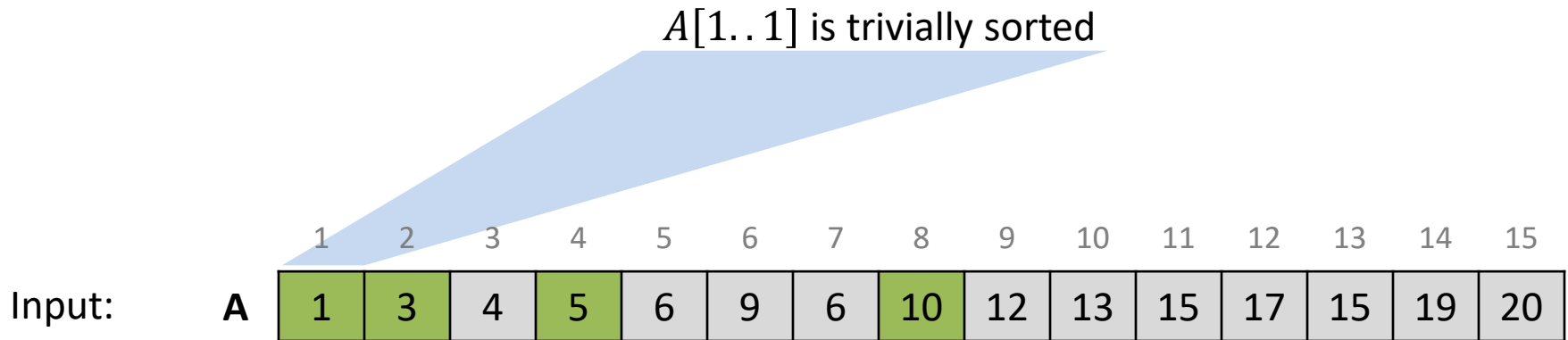
item determined to be at correct sorted location

Quicksort



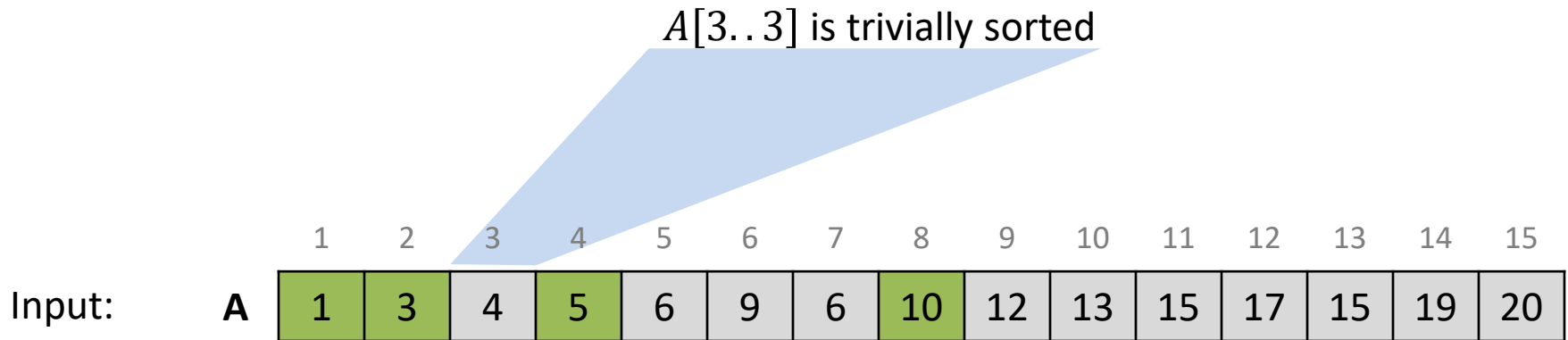
item determined to be at correct sorted location

Quicksort



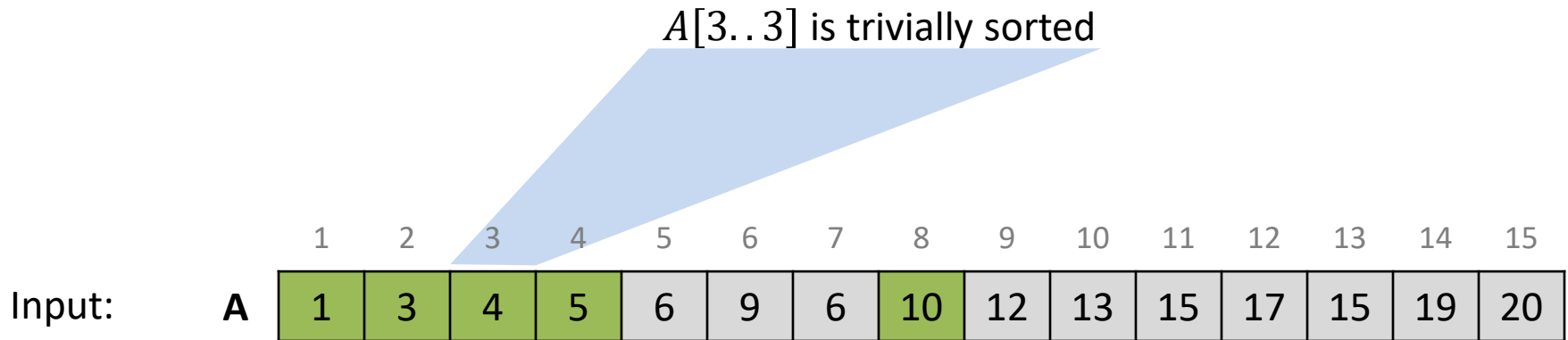
item determined to be at correct sorted location

Quicksort



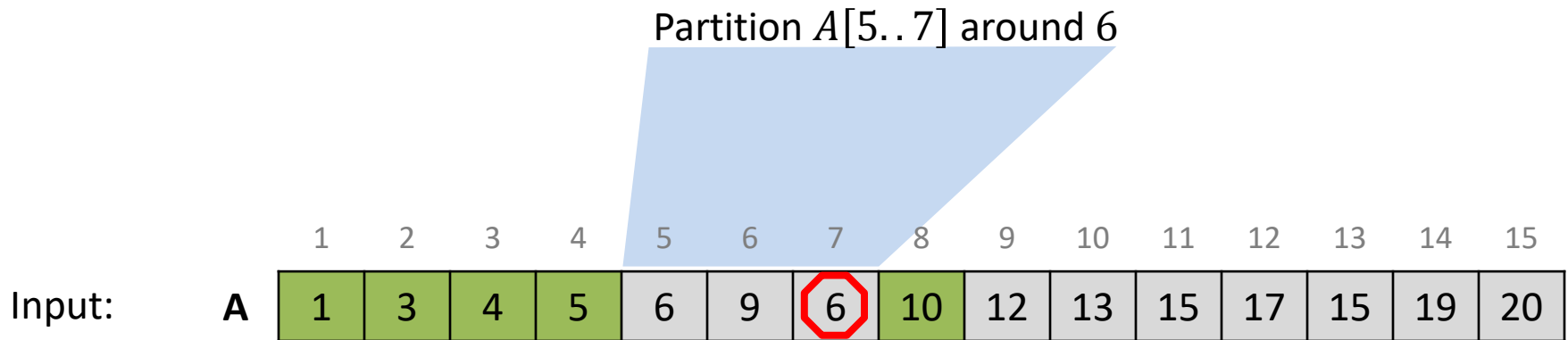
item determined to be at correct sorted location

Quicksort



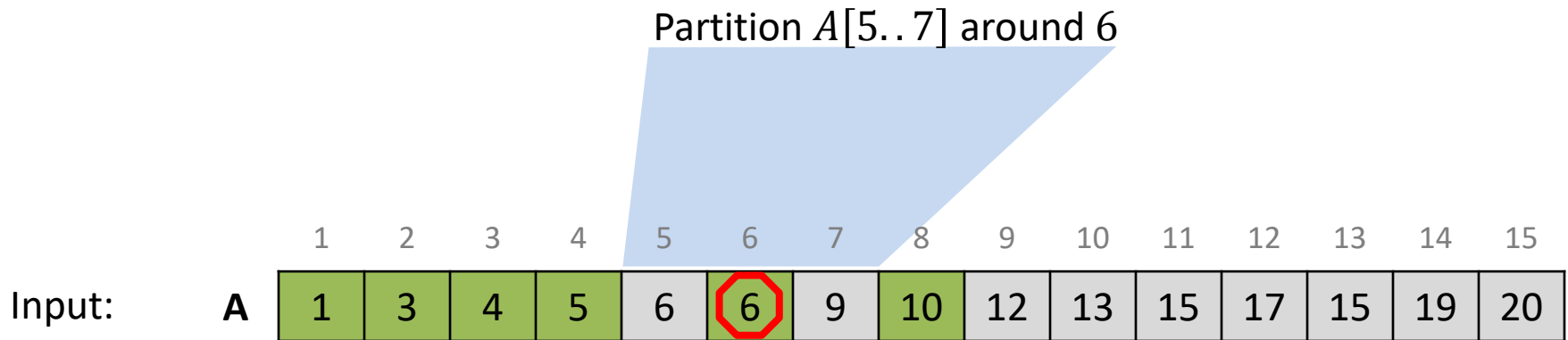
item determined to be at correct sorted location

Quicksort



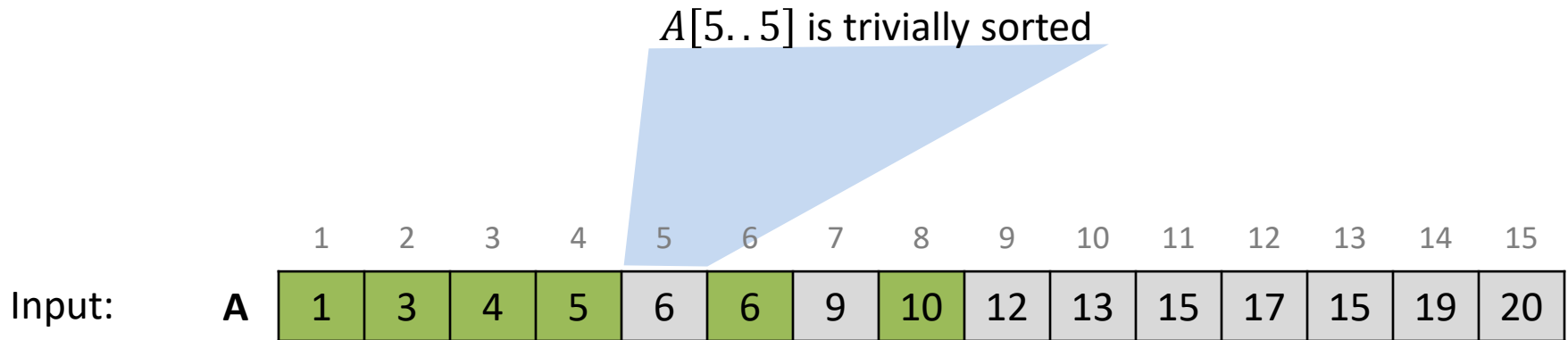
item determined to be at correct sorted location

Quicksort



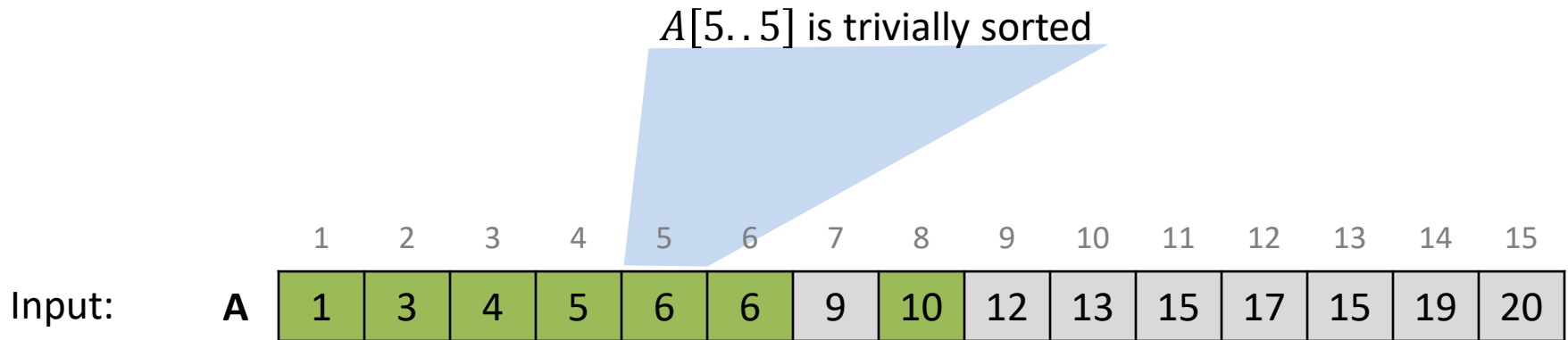
 item determined to be at correct sorted location

Quicksort



item determined to be at correct sorted location

Quicksort



item determined to be at correct sorted location

Quicksort

$A[7..7]$ is trivially sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

$A[7..7]$ is trivially sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..15]$ around 20

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..15]$ around 20

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..14]$ around 19

Input:

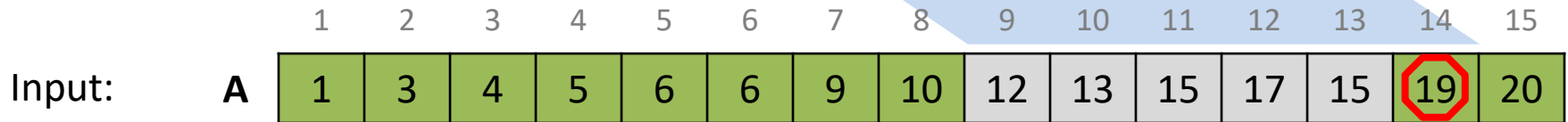
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..14]$ around 19



item determined to be at correct sorted location

Quicksort

Partition $A[9..13]$ around 15

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	17	15	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..13]$ around 15

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..11]$ around 15

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..11]$ around 15

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..10]$ around 13

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

Partition $A[9..10]$ around 13

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

$A[9..9]$ is trivially sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

$A[9..9]$ is trivially sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

$A[13..13]$ is trivially sorted

Input:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

$A[13..13]$ is trivially sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

$A[1..15]$ is now fully sorted

Input:

A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	4	5	6	6	9	10	12	13	15	15	17	19	20



item determined to be at correct sorted location

Quicksort

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ rearranged in non-decreasing order of value.

QUICKSORT (A, p, r)

1. **if** $p < r$ **then**
2. // partition $A[p..r]$ into $A[p..q - 1]$ and $A[q + 1..r]$ such that everything in
 $A[p..q - 1]$ is $\leq A[q]$ and everything in $A[q + 1..r]$ is $\geq A[q]$
3. $q =$ PARTITION (A, p, r)
4. // recursively sort the left part
5. QUICKSORT ($A, p, q - 1$)
6. // recursively sort the right part
7. QUICKSORT ($A, q + 1, r$)

Worst-case Running Time of Quicksort

QUICKSORT (A, p, r)

1. **if** $p < r$ **then**
2. // partition $A[p..r]$ into $A[p..q - 1]$
 and $A[q + 1..r]$ such that everything
 in $A[p..q - 1]$ is $\leq A[q]$ and everything
 in $A[q + 1..r]$ is $\geq A[q]$
3. $q = \text{PARTITION} (A, p, r)$
4. // recursively sort the left part
5. QUICKSORT ($A, p, q - 1$)
6. // recursively sort the right part
7. QUICKSORT ($A, q + 1, r$)

Assuming $n = r - p + 1$, the worst-case running time of quicksort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ \max_{p \leq q \leq r} \{T(q - p) + T(r - q)\} + \Theta(n) & \text{if } n > 1. \end{cases}$$

Replacing q with $k + p - 1$, we get:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ \max_{1 \leq k \leq n} \{T(k - 1) + T(n - k)\} + \Theta(n) & \text{if } n > 1. \end{cases}$$

Worst-case Running Time of Quicksort (Upper Bound)

For $n > 1$ and a constant $c > 0$,

$$T(n) = \max_{1 \leq k \leq n} \{T(k-1) + T(n-k)\} + cn$$

Our guess for upper bound: $T(n) \leq c_1 n^2$ for constant $c_1 > 0$.

Using this bound on the right side of the recurrence equation, we get.

$$\begin{aligned} T(n) &\leq \max_{1 \leq k \leq n} \{c_1(k-1)^2 + c_1(n-k)^2\} + cn \\ \Rightarrow T(n) &\leq c_1 \max_{1 \leq k \leq n} \{(k-1)^2 + (n-k)^2\} + cn \end{aligned}$$

But $(k-1)^2 + (n-k)^2$ reaches its maximum value for $k = 1$ and $k = n$.

Hence,

$$\begin{aligned} T(n) &\leq c_1((1-1)^2 + (n-1)^2) + cn \\ \Rightarrow T(n) &\leq c_1(n-1)^2 + cn \\ \Rightarrow T(n) &\leq c_1 n^2 - (c_1(2n-1) - cn) \end{aligned}$$

Worst-case Running Time of Quicksort (Upper Bound)

But for $c_1 \geq c$, we have,

$$\begin{aligned}c_1(2n - 1) &\geq c(2n - 1) \\ \Rightarrow c_1(2n - 1) &\geq 2cn - c \\ \Rightarrow c_1(2n - 1) - cn &\geq cn - c\end{aligned}$$

But $n \geq 1 \Rightarrow cn \geq c \Rightarrow cn - c \geq 0$, and thus

$$\begin{aligned}c_1(2n - 1) - cn &\geq 0 \\ \Rightarrow -(c_1(2n - 1) - cn) &\leq 0 \\ \Rightarrow c_1n^2 - (c_1(2n - 1) - cn) &\leq c_1n^2\end{aligned}$$

But $T(n) \leq c_1n^2 - (c_1(2n - 1) - cn)$.

Hence, $T(n) \leq c_1n^2$ for $c_1 \geq c$.

Worst-case Running Time of Quicksort (Lower Bound)

For $n > 1$ and a constant $c > 0$,

$$T(n) = \max_{1 \leq k \leq n} \{T(k-1) + T(n-k)\} + cn$$

Our guess for lower bound: $T(n) \geq c_2 n^2$ for constant $c_2 > 0$.

Using this bound on the right side of the recurrence equation, we get.

$$\begin{aligned} T(n) &\geq \max_{1 \leq k \leq n} \{c_2(k-1)^2 + c_2(n-k)^2\} + cn \\ \Rightarrow T(n) &\geq c_2 \max_{1 \leq k \leq n} \{(k-1)^2 + (n-k)^2\} + cn \end{aligned}$$

But $(k-1)^2 + (n-k)^2$ reaches its maximum value for $k = 1$ and $k = n$.

Hence,

$$\begin{aligned} T(n) &\geq c_2((1-1)^2 + (n-1)^2) + cn \\ \Rightarrow T(n) &\geq c_2(n-1)^2 + cn \\ \Rightarrow T(n) &\geq c_2 n^2 + (cn - c_2(2n-1)) \end{aligned}$$

Worst-case Running Time of Quicksort (Lower Bound)

But for $c_2 \leq \frac{c}{2}$, we have,

$$\begin{aligned}c_2(2n - 1) &\leq \frac{c}{2}(2n - 1) \\ \Rightarrow c_2(2n - 1) &\leq cn - \frac{c}{2} \\ \Rightarrow cn - c_2(2n - 1) &\geq \frac{c}{2}\end{aligned}$$

But $c > 0$, and thus

$$\begin{aligned}cn - c_2(2n - 1) &> 0 \\ \Rightarrow c_2n^2 + (cn - c_2(2n - 1)) &> c_2n^2\end{aligned}$$

But $T(n) \geq c_2n^2 + (cn - c_2(2n - 1))$.

Hence, $T(n) \geq c_2n^2$ for $c_2 \leq \frac{c}{2}$.

Worst-case Running Time of Quicksort (Tight Bound)

We have proved that

$$T(n) \leq c_1 n^2 \text{ for } c_1 \geq c,$$

$$\text{and } T(n) \geq c_2 n^2 \text{ for } c_2 \leq \frac{c}{2}.$$

Thus $c_2 n^2 \leq T(n) \leq c_1 n^2$ for constants $c_1 \geq c$ and $c_2 \leq \frac{c}{2}$.

Hence, $T(n) = \Theta(n^2)$.

Average Case Running Time of Quicksort

Average Case Running Time of Quicksort

QUICKSORT (A, p, r)

1. **if** $p < r$ **then**
2. // partition $A[p..r]$ into $A[p..q-1]$
 and $A[q+1..r]$ such that everything
 in $A[p..q-1]$ is $\leq A[q]$ and everything
 in $A[q+1..r]$ is $\geq A[q]$
3. $q = \text{PARTITION} (A, p, r)$
4. // recursively sort the left part
5. QUICKSORT ($A, p, q-1$)
6. // recursively sort the right part
7. QUICKSORT ($A, q+1, r$)

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ \frac{1}{n} \sum_{1 \leq k \leq n} \{T(k-1) + T(n-k)\} + \Theta(n) & \text{if } n > 1. \end{cases}$$

Average Case Running Time of Quicksort

For $n > 1$ and a constant $c > 0$,

$$\begin{aligned}T(n) &= \frac{1}{n} \sum_{1 \leq k \leq n} \{T(k-1) + T(n-k)\} + cn \\ \Rightarrow nT(n) &= \sum_{1 \leq k \leq n} \{T(k-1) + T(n-k)\} + cn^2 \\ \Rightarrow nT(n) &= 2 \sum_{0 \leq k \leq n-1} T(k) + cn^2 \quad \dots (1)\end{aligned}$$

Replacing n with $n-1$,

$$\Rightarrow (n-1)T(n-1) = 2 \sum_{0 \leq k \leq n-2} T(k) + c(n-1)^2 \quad \dots (2)$$

Subtracting equation (2) from equation (1), we get

$$\begin{aligned}nT(n) - (n-1)T(n-1) &= 2T(n-1) + c(2n-1) \\ \Rightarrow nT(n) - (n+1)T(n-1) &= c(2n-1)\end{aligned}$$

Dividing both sides by $n(n+1)$, we get

$$\frac{T(n)}{n+1} - \frac{T(n-1)}{n} = \frac{c(2n-1)}{n(n+1)}$$

Average Case Running Time of Quicksort

Assuming $\frac{T(n)}{n+1} = A(n)$, we get from the equation from the previous slide,

$$A(n) - A(n-1) = \frac{c(2n-1)}{n(n+1)}$$

$$\Rightarrow A(n) = A(n-1) + \frac{c(2n-1)}{n(n+1)}$$

$$\Rightarrow A(n) = A(n-1) + \frac{2c}{n+1} - \frac{c}{n(n+1)}$$

$$\Rightarrow A(n) < A(n-1) + \frac{2c}{n+1}$$

$$\Rightarrow A(n) < A(n-2) + \frac{2c}{n} + \frac{2c}{n+1}$$

$$\Rightarrow A(n) < A(n-3) + \frac{2c}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}$$

$$\Rightarrow A(n) < A(n-k) + \frac{2c}{n-k+2} + \frac{2c}{n-k+3} + \dots + \frac{2c}{n} + \frac{2c}{n+1}$$

$$\Rightarrow A(n) < A(1) + \frac{2c}{3} + \frac{2c}{4} + \dots + \frac{2c}{n} + \frac{2c}{n+1}$$

Average Case Running Time of Quicksort

Since $A(1) = \frac{T(1)}{2} = \Theta(1)$, we get,

$$\Rightarrow A(n) < \Theta(1) + 2c \left(\frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n} + \frac{1}{n+1} \right)$$

$$\Rightarrow A(n) < \Theta(1) + 2c \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} + \frac{1}{n+1} \right) - 2c \left(1 + \frac{1}{2} \right)$$

But $H_{n+1} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} + \frac{1}{n+1}$ is the $n + 1$ 'st *Harmonic Number*,

and $\lim_{n \rightarrow \infty} H_{n+1} = \ln(n + 1) + \gamma$, where $\gamma \approx 0.5772$ is known as the

Euler-Mascheroni constant.

Hence, for $n \rightarrow \infty$: $A(n) < 2c(\ln(n + 1) + \gamma) - 3c + \Theta(1)$

$$\Rightarrow A(n) < 2c \ln(n + 1) + \Theta(1)$$

$$\Rightarrow \frac{T(n)}{n+1} < 2c \ln(n + 1) + \Theta(1)$$

$$\Rightarrow T(n) < 2c (n + 1) \ln(n + 1) + \Theta(n)$$

$$\Rightarrow T(n) = O(n \log n)$$

[Optional]

Proof of Correctness of Partition

Correctness of Partition

Input: A subarray $A[p : r]$ of $r - p + 1$ numbers, where $p \leq r$.

Output: Elements of $A[p : r]$ are rearranged such that for some $q \in [p, r]$ everything in $A[p : q - 1]$ is $\leq A[q]$ and everything in $A[q + 1 : r]$ is $\geq A[q]$. Index q is returned.

PARTITION (A, p, r)

1. $x = A[r]$
2. $i = p - 1$
3. **for** $j = p$ **to** $r - 1$
4. **if** $A[j] \leq x$
5. $i = i + 1$
6. exchange $A[i]$ with $A[j]$
7. exchange $A[i + 1]$ with $A[r]$
8. **return** $i + 1$

Loop Invariant

At the start of each iteration of the **for** loop of lines 3–6, for any array index k ,

1. **if** $p \leq k \leq i$,
 then $A[k] \leq x$.
2. **if** $i + 1 \leq k \leq j - 1$,
 then $A[k] > x$.
3. **if** $k = r$,
 then $A[k] = x$.

*When the math
gets tough*

*Analysis of Algorithms
in
Rhymes and Pictures*

ChatGPT 4o mini



Quicksort: A Rhyming Tale

When sorting fast is what you need,
Quicksort's the one that takes the lead.
It picks a pivot, quick and slick,
Then splits the list—watch the trick!

To the left go numbers small,
To the right, the big ones all.
It sorts the halves, again, again,
Till every piece falls into place then.

Divide and conquer's how it goes,
In no time flat, your list will close.
It's quick by name, and quick by might,
For sorting, it's a shining light!



Complexity Analysis in Rhyme

For the **Best Case**, when luck's on your side,
 $O(n \log n)$, Quicksort will glide.
It splits things neatly, half and half,
Making sorting a breezy path.

The **Average Case** is just the same,
 $O(n \log n)$ still holds its fame.
Though splitting might not be exact,
It's still quite fast—that's a fact!

But the **Worst Case**, oh, beware,
When pivots pick with little care.
If the list splits bad and long,
 $O(n^2)$ comes along!

But still, it's fast and tends to win,
With clever picks, it works within,
A sorting gem, both sharp and bright,
Quickly putting things just right!
To the left go numbers small,

